

Angelernt oder Autodidakt?

Supervised Learning vs. Self-Play Reinforcement Learning mit MCTS in Schachagenten

Danny Hoffmann Tim Lehmann Joshua Meyer Philipp Meyer

Modul: Projekt Data Science und Künstliche Intelligenz
Kurs: WDSKI 23 B
Dozent: Prof. Dr. Maximilian Scherer
Abgabe: 30. Juli 2026



Mai – Juli 2026

Inhaltsverzeichnis

1	Einleitung	2
2	Theoretische Grundlagen	2
2.1	Schach als formales Problem	2
2.2	Spielstärke quantifizieren	5
2.3	Klassische Engines: Minimax, Alpha-Beta, Stockfish/NNUE	6
2.4	Brettrepräsentation & Zugkodierung	7
2.5	Neuronale Netze für Schach: CNN/ResNet, Policy- und Value-Head	8
2.6	Supervised Learning im Schachkontext	9
2.7	Monte Carlo Tree Search (UCT, PUCT)	10
2.8	AlphaZero-Paradigma: Self-Play Reinforcement Learning	12
3	Implementierung	14
3.1	Modellarchitektur	14
3.2	Supervised Learning: Datenpipeline, Modell, Training	16
3.3	Reinforcement Learning: MCTS-Kern, Self-Play, Trainings-Loop	18
4	Evaluation & Ergebnisse	19
4.1	Versuchsaufbau	19
4.2	Ergebnisse Supervised Learning	20
4.3	Ergebnisse Reinforcement Learning	24
4.4	Direktvergleich: SL vs. RL	27
4.5	Rechenaufwand & Inferenzzeit	28
5	Diskussion	30
6	Fazit	31
	Literatur	32

1 Einleitung

Um ein neuronales Netz Schach spielen zu lassen, gibt es zwei grundsätzlich verschiedene Wege, um an Trainingsdaten zu gelangen. Entweder lernt das Netz aus bereits vorhandenen Partien und ahmt die dort getroffenen Entscheidungen nach, oder es erzeugt seine Daten selbst, indem es gegen sich selbst spielt und aus dem Ausgang dieser Partien lernt. Beide Wege ergeben ein Netz, das eine Stellung auf eine Zugpräferenz und eine Bewertung abbildet, und beide nutzen dieselbe Suche, um daraus Züge auszuwählen.

Welcher der beiden Wege der bessere ist, gilt seit AlphaZero als weitgehend beantwortet. Ein ausschließlich im Self-Play trainiertes Netz übertraf die damals stärksten Programme, ohne je eine menschliche Partie gesehen zu haben [1]. Diese Antwort setzt allerdings einen Rechenaufwand voraus, der für die Datenerzeugung mehrere tausend TPUs parallel beschäftigt. Unter deutlich kleineren Verhältnissen ist die Frage erneut offen, denn Self-Play muss seine Trainingsdaten erst selbst erzeugen, während Supervised Learning auf einem bereits vorhandenen Korpus aufsetzt. Genau diese Verschiebung untersucht die vorliegende Arbeit. Sie fragt, welches Trainingssignal den stärkeren Agenten liefert, wenn die verfügbare Rechenzeit nicht die eines Rechenzentrums ist, sondern die einzelner GPU-Stunden auf frei verfügbaren Plattformen.

Dazu wurden zwei Agenten entwickelt, die sich ausschließlich im Trainingssignal unterscheiden. Sie teilen sich Netzarchitektur, Zustandskodierung und Monte Carlo Tree Search, sodass ein Vergleich zwischen ihnen allein das Trainingsparadigma misst. Der per Supervised Learning trainierte Agent lernt aus menschlichen Partien, Taktikaufgaben und Endspieldatenbanken, der per Reinforcement Learning trainierte erzeugt seine Daten im Self-Play nach dem AlphaZero-Verfahren. Im Folgenden werden beide kurz als SL-Agent und RL-Agent bezeichnet. Sie werden anschließend in Matches gegeneinander und gegen Stockfish [2] auf einer gemeinsamen Elo-Skala eingeordnet.

2 Theoretische Grundlagen

2.1 Schach als formales Problem

Bevor Verfahren zur Zugwahl verglichen werden können, muss präzisiert werden, welches Problem sie eigentlich lösen.

Spieltheoretische Einordnung Schach ist ein endliches, deterministisches Zwei-Personen-Nullsummenspiel mit perfekter Information und alternierenden Zügen. Jede dieser Eigenschaften hat unmittelbare Konsequenzen für die in dieser Arbeit betrachteten Verfahren:

- **Perfekte Information und Determinismus** Beiden Spielern ist zu jedem Zeitpunkt der vollständige Spielzustand bekannt, und jeder Zug führt genau zu einer Folgestellung. Der Spielbaum ist damit von jeder Stellung aus prinzipiell vollständig

expandierbar. Anders als etwa beim Poker müssen keine Annahmen über verdeckte Information getroffen werden und anders als beim Backgammon verbreitern keine Zufallsknoten den Baum.

- **Nullsumme** Der Gewinn der einen Seite ist exakt der Verlust der anderen. Eine einzige Bewertungsgröße aus Sicht der Seite am Zug genügt daher, um die Präferenzen beider Spieler zu beschreiben. Das rechtfertigt sowohl das Minimax-Prinzip (Abschnitt 2.3) als auch die Eigenschaft, dass das Netz mit einem einzigen Value-Head auskommt (Abschnitt 2.5).
- **Endlichkeit** Die 50-Züge-Regel (eine Partie ist Remis wenn 50 Züge lang kein Bauer bewegt und keine Figur geschlagen wurde) und die Stellungswiederholungsregel (eine Partie ist Remis wenn dreimal die gleiche Stellung auf dem Brett steht) stellen sicher, dass jede Partie nach endlich vielen Zügen terminiert. Der Ausgang ist eines von drei Ergebnissen, formal $z \in \{1, \frac{1}{2}, 0\}$ für Sieg, Remis und Niederlage (kurz WDL).
- **Alternierende Züge** Es gibt keine simultanen Entscheidungen. Zusammen mit perfekter Information folgt daraus, dass für beide Seiten eine optimale Strategie existiert.

Von den Spielregeln selbst sind für das Weitere vor allem die Sonderzüge Rochade, en-passant und Bauernumwandlung sowie die Remisregeln relevant. Sie erscheinen in Abschnitt 2.4 als eigene Ebenen des Eingabetensors bzw. als eigene Gruppe im Ausgaberaum wieder.

Zustand, Zug und Spielbaum Eine Stellung s umfasst mehr als die Belegung der 64 Felder. Zum Zustand gehören zusätzlich die Seite am Zug, die vier Rochaderechte, ein etwaiges en-passant-Zielfeld und der Halbzugzähler der 50-Züge-Regel. Genau dieses Tupel hält die Forsyth-Edwards-Notation (FEN) fest, und genau diese Bestandteile tauchen in Tabelle 1 als Ebenen des Brett-Tensors wieder auf.

Diese Zustandsdefinition ist so gewählt, dass sie die Markov-Eigenschaft erfüllt. Sowohl die Menge der legalen Fortsetzungen als auch der spieltheoretische Wert einer Stellung hängen ausschließlich von s ab und nicht davon, über welche Zugfolge s erreicht wurde. Zwei durch unterschiedliche Zugreihenfolgen entstandene, ansonsten identische Stellungen (Transpositionen) sind für die Zugwahl dasselbe Objekt. Streng genommen bildet die Regel zur dreifachen Stellungswiederholung hiervon eine Ausnahme, da sie auf die Partiehistorie zurückgreift. Sie wird in der Suche gesondert behandelt und geht nicht in die Stellungsbewertung ein. Diese Eigenschaft ist die Grundlage für die in Abschnitt 2.4 gewählte historienfreie Kodierung.

Bezeichnet $A(s)$ die Menge der in s legalen Züge und $s \cdot a$ die Stellung, die aus s durch Ausführung von $a \in A(s)$ hervorgeht, so lässt sich das gesamte Spiel als Baum darstellen: Die Wurzel ist die Ausgangsstellung, jede Kante entspricht einem Halbzug, die Knoten einer Ebene gehören abwechselnd der einen und der anderen Seite, und die Blätter sind

terminale Stellungen mit bekanntem Ergebnis $z(s)$. Sowohl die klassische Alpha-Beta-Search (Abschnitt 2.3) als auch die Monte Carlo Tree Search (Abschnitt 2.7) operieren auf genau diesem Baum. Sie unterscheiden sich lediglich darin, welchen Ausschnitt davon sie betrachten.

Theoretisch lösbar, praktisch unlösbar Fasst man den Zustand s so weit, wie es Shannon [3] für sein Stellungsformat vorsieht (Figurenstellung, Zugrecht, Rochaderechte, letzter Zug sowie der Zähler seit dem letzten Bauernzug oder Schlagfall), so besitzt jede Stellung genau einen der drei Werte *gewonnen für Weiß*, *remis* oder *gewonnen für Schwarz*.

Da jede Partie nach endlich vielen Zügen endet, lässt sich der Wert einer Stellung bestimmen, indem man vom Partieende rückwärts arbeitet [3]. Die Kombinatorik macht diesen Weg unbegehrbar. Shannon rechnet mit etwa 30 legalen Zügen pro Stellung, also rund 10^3 Fortsetzungen je Zugpaar, und bei einer typischen Partielänge von 40 Zügen mit etwa 10^{120} zu prüfenden Varianten. Eine Maschine, die eine Variante pro Mikrosekunde abarbeitet, bräuhete nach Shannon also über 10^{90} Jahre für den ersten Zug.

Zwei Beispiele verdeutlichen noch einmal die Komplexität. Erstens sind Endspieldatenbanken für Schach zwar exakt gelöst, aber nur bis sieben Figuren auf dem Brett. Allein diese Tabellen belegen rund 18 TB. Zweitens gelang mit dem Spiel Dame im Jahr 2007 die vollständige Lösung eines vergleichbaren Spiels. Bei einem Suchraum von etwa $5 \cdot 10^{20}$ Stellungen konnte gezeigt werden, dass beidseitig optimales Spiel zum Remis führt [4]. Der Schachsuchraum liegt mehr als zwanzig Größenordnungen darüber.

Was ist ein guter Zug? Aus dieser Lücke zwischen theoretischer Bestimmtheit und praktischer Unberechenbarkeit ergibt sich eine wichtige Fragestellung dieser Arbeit: "*Was ist ein guter Zug?*". Ein Zug ist *objektiv* optimal, wenn er den spieltheoretischen Wert der Stellung nicht verschlechtert. Da dieser theoretische Wert aber unzugänglich ist, lässt sich diese Eigenschaft für keinen konkreten Zug nachweisen. In der Praxis wird ein Zug daher *operativ* bewertet. Ein Zug ist gut, wenn er unter gegebener Rechenzeit die Gewinnerwartung gegen einen realen Gegner maximiert.

Für die Zugwahl durch Engines im Schach gibt es im Wesentlichen zwei Stellschrauben: *wie viele* Stellungen ein Programm betrachtet (Suche) und *wie gut* es eine einzelne Stellung einschätzt (Bewertung). Klassische Engines setzen primär auf die erste Stellschraube und kompensieren eine vergleichsweise grobe Bewertungsfunktion durch enorme Suchbreite (Abschnitt 2.3). Der in dieser Arbeit verfolgte AlphaZero-artige Ansatz verschiebt das Gewicht auf die zweite: Ein aufwändiges neuronales Netz liefert Bewertung und Zugvorauswahl, sodass mit deutlich weniger besuchten Stellungen ausgekommen wird (Abschnitte 2.5–2.7).

Eine unmittelbare Konsequenz dieser Sichtweise ist, dass Spielstärke nicht absolut, sondern nur *relativ* messbar ist, und zwar über Partieergebnisse gegen andere Spieler. Der nächste

Abschnitt stellt mit dem Elo-Modell das Verfahren vor, mit dem diese relativen Ergebnisse in eine vergleichbare Skala überführt werden.

2.2 Spielstärke quantifizieren

Das gebräuchliche Verfahren hierfür ist das Elo-System [5]. Es ordnet jedem Spieler eine Zahl R zu und modelliert den erwarteten Punktanteil E_A von Spieler A gegen Spieler B allein als Funktion der Differenz beider Zahlen:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}.$$

Die Skalierung ist so gewählt, dass 400 Punkte Vorsprung einem erwarteten Punktanteil von etwa 0,91 entsprechen, also einem Verhältnis von rund 10 : 1. Ein Remis zählt dabei als halber Punkt.

Für die Evaluation in Kapitel 4 wird die Beziehung invertiert: Aus dem in einem Match erzielten Punktanteil $S \in (0, 1)$ folgt die geschätzte Elo-Differenz zum Gegner als

$$\Delta R = 400 \cdot \log_{10} \left(\frac{S}{1 - S} \right).$$

Ist die Stärke des Gegners bekannt, ergibt sich daraus ein absoluter Wert. Das Elo-System liefert also stets nur Differenzen und benötigt einen Anker. In dieser Arbeit dienen dafür Stockfish-Skill-Level mit bekannter Referenzstärke.

Als Einschränkung ist zu beachten, dass ΔR eine Schätzung aus einer endlichen Stichprobe ist.

Elo misst Spielstärke ausschließlich aggregiert über Partieausgänge und benötigt dafür viele Partien. Für eine granularere Bewertung greifen wir zusätzlich auf eine Kennzahl zurück, die jeden einzelnen Zug beurteilt und auf der Stellungsbewertung einer starken Referenzengine beruht. Eine solche Engine drückt den Vorteil einer Seite als eine einzige Zahl aus, die konventionell in *Centipawns* (cp) angegeben wird. Ein Centipawn entspricht einem Hundertstel Bauern, sodass 100 cp den Wert eines Bauern haben [6]. Wie diese Bewertung zustande kommt, beschreibt Abschnitt 2.3.

Der *Centipawn-Loss* eines Zuges ist die nichtnegative Differenz zwischen der Bewertung des von der Referenzengine bevorzugten Zuges und der des tatsächlich gespielten Zuges. Spielt ein Spieler den besten Zug, ist sein Centipawn-Loss null. Jede Abweichung verschlechtert die Stellung um einen in Centipawns messbaren Betrag. Gemittelt über alle Züge einer Partie ergibt sich der *durchschnittliche Centipawn-Loss* (average centipawn loss, ACPL) als Maß der Zuggenauigkeit. Je kleiner der Wert, desto näher spielt ein Agent am Referenzspiel [7]. Das Paper von Guid und Bratko [7] stellt das methodische Verfahren vor, aber nicht unter dem Namen *Centipawn-Loss*. Anders als Elo liefert diese Kennzahl bereits aus wenigen Partien ein dichtes Signal, da jeder Zug einen Messpunkt beiträgt.

Beide Maße hängen eng zusammen. Leite und Oliveira [8] werten rund 7 800 klassische Partien von zehn Großmeistern (Elo etwa 2200–2700) aus und zeigen, dass der durchschnittliche Centipawn-Loss mit steigender Spielstärke systematisch fällt und stark mit der Elo-Zahl korreliert. Der Centipawn-Loss lässt sich damit als per-Zug-Gegenstück zu Elo verstehen. Elo schätzt Stärke relativ aus Partieergebnissen, der Centipawn-Loss aus der Qualität der einzelnen Züge gegen eine feste Referenz. Zu beachten ist jedoch, dass der Centipawn-Loss die Übereinstimmung mit der Referenzengine misst (in dieser Arbeit Stockfish) und nicht die Nähe zum wahren spieltheoretischen Wert, der nach Abschnitt 2.1 unzugänglich bleibt. Ein niedriger Centipawn-Loss bedeutet also „spielt wie Stockfish“, nicht „spielt objektiv optimal“.

2.3 Klassische Engines: Minimax, Alpha-Beta, Stockfish/NNUE

Klassische Schachprogramme behandeln die Zugwahl als Suche im Spielbaum (Abschnitt 2.1). Da dieser Baum praktisch nicht vollständig durchsucht werden kann, begrenzt man die Suchtiefe und bewertet die so erreichten Blattstellungen mit einer heuristischen Bewertungsfunktion $\text{eval}(s)$, die Merkmale wie Material, Königssicherheit und Bauernstruktur zu einer einzigen Zahl verrechnet [3].

Der Wert einer Stellung ergibt sich rekursiv nach dem Minimax-Prinzip. Die Seite am Zug wählt den für sie besten Zug, der Gegner den für ihn besten. Bezeichnet $s \cdot a$ die aus s durch Zug a entstehende Stellung, so gilt

$$V(s) = \begin{cases} \text{eval}(s) & \text{Tiefenlimit erreicht,} \\ \max_a V(s \cdot a) & \text{Seite am Zug,} \\ \min_a V(s \cdot a) & \text{Gegner am Zug.} \end{cases}$$

An der Wurzel wird der Zug mit dem höchsten Wert gespielt.

Naiv bewertet Minimax exponentiell viele Stellungen. Alpha-Beta-Pruning liefert dasselbe Ergebnis, überspringt aber Teilbäume, die den Wurzelwert nicht mehr verändern können, weil eine Seite dort bereits einen ausreichend guten Alternativzug kennt. Bei guter Zugsortierung sinkt der effektive Verzweigungsgrad von b auf etwa $\sqrt{b}$, was bei gleichem Aufwand annähernd die doppelte Suchtiefe erlaubt [9]. Auf dieser Grundlage gelang mit Deep Blue 1997 der erste Sieg eines Computers gegen einen amtierenden Schachweltmeister [10].

Moderne Spitzenprogramme wie Stockfish kombinieren eine stark optimierte Alpha-Beta-Suche mit einer neuronalen Bewertungsfunktion (NNUE), die die früher handgeschriebene Heuristik ersetzt [11]. In dieser Arbeit dient Stockfish als Referenzgegner und Elo-Anker (Kapitel 4).

Allen klassischen Verfahren ist gemein, dass die Spielstärke primär aus breiter Suche entsteht, während die Bewertung einer einzelnen Stellung vergleichsweise günstig bleibt. Der hier verfolgte Ansatz verlagert diesen Schwerpunkt. Ein aufwändiges neuronales

Netz übernimmt Bewertung und Zugvorauswahl, sodass deutlich weniger Stellungen durchsucht werden müssen (Abschnitte 2.5–2.7).

2.4 Brettrepräsentation & Zugkodierung

Beide Agenten nutzen dieselbe Kodierung, die eine Schachstellung in einen Eingabetensor fester Größe und jeden Zug in einen Ausgabeindex übersetzt. Das Brett wird dabei stets aus Sicht der Seite am Zug kodiert. Ist Schwarz am Zug, wird es gespiegelt, sodass das Netz nur eine Perspektive lernen muss. Repräsentation und Zugkodierung folgen dem AlphaZero-Schema [1].

Brett-Tensor Eine Stellung wird als Tensor der Form $8 \times 8 \times 20$ dargestellt. Die ersten zwölf Ebenen kodieren die Figuren beider Seiten (je eine Ebene pro Figurentyp), acht weitere die Zusatzinformationen einer Stellung, und eine konstante Ebene liefert eine positionsunabhängige Bias-Information. Tabelle 1 listet die Belegung der Ebenen auf.

Ebene(n)	Inhalt
0–5	eigene Figuren (Bauer, Springer, Läufer, Turm, Dame, König)
6–11	gegnerische Figuren (gleiche Reihenfolge)
12–13	eigene Rochaderechte (Damen-/Königsflügel)
14–15	gegnerische Rochaderechte
16	Farbe am Zug (1, falls im Original Schwarz am Zug)
17	Halbzugzähler /100 (50-Züge-Regel)
18	En-passant-Zielfeld (1 auf dem Zielfeld, sonst 0)
19	konstante Ebene (Wert 1)

Tabelle 1: Die 20 Ebenen des Brett-Tensors $8 \times 8 \times 20$.

Die zwölf Figurenebenen sind rein binär, sie tragen je Feld eine Eins, wenn dort die betreffende Figur steht, und sonst eine Null. Eine solche Ebene wird als Bitboard bezeichnet. Andere Sonderebenen ohne Zielfeld sind entweder überall Null oder Eins. Abbildung 1 zeigt das beispielhaft für die Bauern beider Seiten.

Kodiert wird ausschließlich die aktuelle Stellung, ohne Zughistorie. Die Gründe für diese Abweichung von AlphaZero [1] und Leela [12] werden in Abschnitt 3.1 erläutert.

Zugkodierung Ein Zug wird als Paar (Startfeld, Ebene) kodiert, wobei jedem der 64 Felder 73 Ebenen zugeordnet sind. Die 73 Ebenen gliedern sich in drei Gruppen.

- **Ebenen 0–55, Damenzüge (Sliding).** Acht Himmelsrichtungen $\times$ sieben Distanzen (1–7 Felder). Diese Gruppe deckt alle Zieh- und Schlagzüge von Dame, Turm, Läufer, König und Bauer ab. Auch die Umwandlung in eine Dame wird als gewöhnlicher Zug in diese Ebenen kodiert.

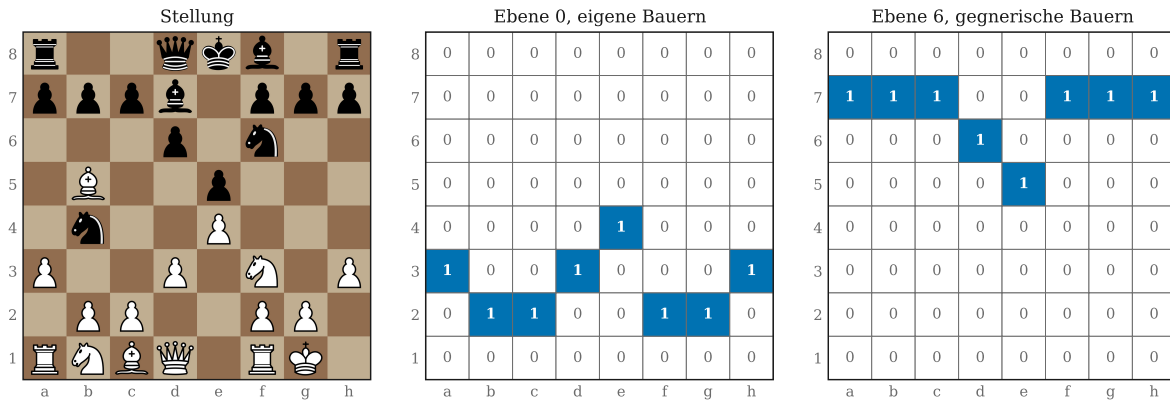


Abbildung 1: Stellung und die zugehörigen Bitboards der Bauern beider Seiten.

- **Ebenen 56–63, Springerzüge.** Die acht festen Sprungoffsets.
- **Ebenen 64–72, Unterverwandlungen.** Drei Figuren (Springer, Läufer, Turm) $\times$ drei Richtungen (geradeaus, Schlag links/rechts).

Das ergibt einen Ausgaberaum von $8 \times 8 \times 73 = 4672$ Zellen. Von diesen können jedoch nur 1858 jemals einem legalen Zug entsprechen, denn viele Paare liegen außerhalb des Bretts (etwa ein Springersprung vom Rand). Eine feste Lookup-Tabelle bildet daher jeden Zug auf genau einen von 1858 Ausgabeindizes ab. Vor der Softmax-Normierung werden die in der jeweiligen Stellung illegalen Indizes maskiert.

2.5 Neuronale Netze für Schach: CNN/ResNet, Policy- und Value-Head

Eine Schachstellung liegt bereits als zweidimensionales 8×8 -Raster vor und ähnelt damit einem Bild. Faltungsnetze (CNNs) sind für solche gitterförmigen Eingaben besonders geeignet. Ihre Filter erkennen lokale Muster, etwa Bauernketten oder die Königsstellung, unabhängig davon, wo auf dem Brett sie auftreten, und teilen ihre Gewichte über alle Felder. Der in Abschnitt 2.4 beschriebene Eingabetensor $8 \times 8 \times 20$ dient dabei als „Bild“ mit 20 Kanälen.

Um Stellungen ausreichend tief zu verarbeiten, werden viele Faltungsschichten gestapelt. Sehr tiefe Netze leiden jedoch unter Vanishing Gradients. Residual-Netze (ResNets) begegnen dem mit Skip Connections, die den Eingang eines Blocks direkt zu dessen Ausgang addieren, sodass der Block nur noch die Abweichung (das Residuum) lernen muss [13]. Die für Schach etablierte und auch hier verwendete Architektur von Leela Chess Zero besteht aus einem solchen Residual-Tower [12, 11].

Anders als klassische ResNet-Blöcke setzt Leela auf Squeeze-and-Excitation-Blöcke (SE), die einen Aufmerksamkeitsmechanismus über die Kanäle ergänzen. Aus dem globalen Mittel jeder Feature Map wird ein Gewicht je Kanal bestimmt, mit dem die Kanäle

anschließend neu skaliert werden. So kann das Netz kontextabhängig betonen, welche Merkmale in einer Stellung wichtig sind [14]. Leela nutzt dabei eine erweiterte, multiplikativ-additive Variante.

Auf den gemeinsamen Rumpf folgen zwei Heads (Abbildung 2). Der Policy-Head gibt für jeden der 1858 möglichen Züge (Abschnitt 2.4) einen unnormierten Wert (Logit) aus und bildet damit die Zugpräferenz. Der Value-Head schätzt den Ausgang der Partie. Statt eines einzelnen skalaren Wertes verwendet die betrachtete Architektur eine WDL-Ausgabe, also drei über Softmax normierte Wahrscheinlichkeiten für Sieg, Remis und Niederlage aus Sicht der Seite am Zug. Da beide Heads denselben Rumpf teilen, beruhen Bewertung und Zugauswahl auf einer gemeinsamen Merkmalsbasis.

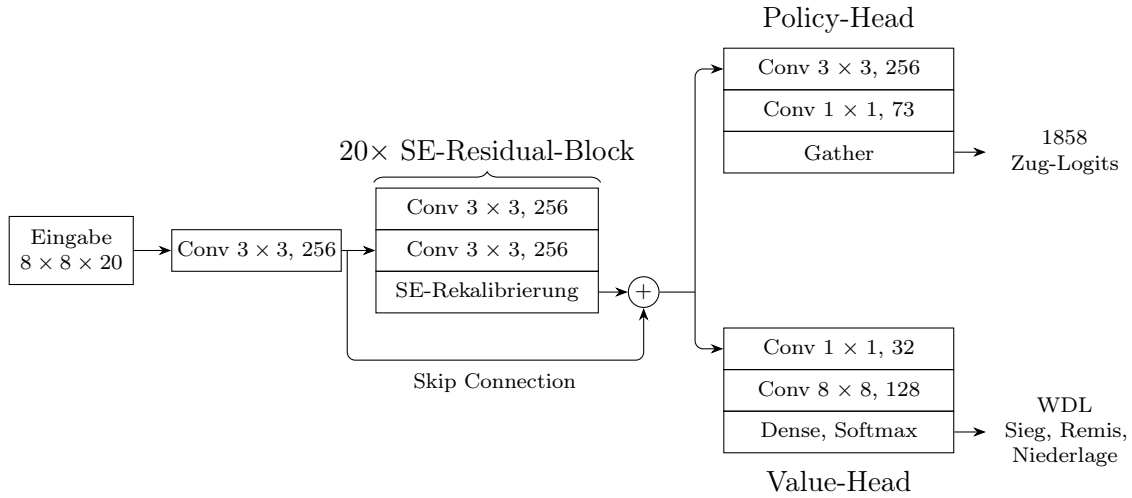


Abbildung 2: Leela-Architektur mit geteiltem Rumpf, Policy- und Value-Head.

Die Abbildung nennt bereits die hier verwendete Konfiguration, also 20 Blöcke mit 256 Feature Maps. Batch-Normierung und ReLU sind darin nicht eigens dargestellt. Abschnitt 3.1 begründet diese Wahl und benennt die Abweichungen von Leela.

2.6 Supervised Learning im Schachkontext

Beim Supervised Learning lernt das Netz die Abbildung einer Stellung auf ihre Zielwerte, also eine Zugverteilung (Policy) und eine Bewertung (Value), aus einem festen, bereits beschrifteten Datensatz. Anders als Reinforcement Learning (Abschnitt 2.8), das seine Trainingsdaten selbst erzeugt, ahmt ein so trainiertes Netz die im Datensatz enthaltenen Entscheidungen nach. Seine Spielstärke ist dadurch grundsätzlich durch die Qualität der Daten begrenzt, denn das Netz kann seine Lehrerquelle nachbilden, sie aber kaum übertreffen. Prominent ist dieser Ansatz aus AlphaGo, dessen Policy-Netz zunächst per Supervised Learning aus menschlichen Meisterpartien lernte [15].

Für beide Heads werden Zielwerte benötigt. Das Policy-Ziel kann der tatsächlich gespielte Zug sein (eine einzelne „richtige“ Antwort) oder eine ganze Zugverteilung, etwa aus den Analysen einer Engine. Das Value-Ziel beschreibt den Ausgang der Stellung. Dabei stehen sich zwei Signale gegenüber, nämlich der Ausgang der gesamten Partie, billig zu erheben, aber verrauscht, da jede Stellung einer gewonnenen Partie pauschal als „Sieg“ gilt, und die genauere, aber rechenintensive Einzelbewertung durch eine starke Engine. Diese Unterscheidung wird in Abschnitt 3.2 vertieft.

Ebenso entscheidend ist, woher die Stellungen stammen. Menschliche Partiedatenbanken sind umfangreich und stilistisch vielfältig, enthalten aber Fehler und menschliche Eigenheiten. Engine-Partien sind stärker und konsistenter, dafür einseitiger und teurer zu erzeugen. Auch die Zusammensetzung des Datensatzes prägt, was das Netz lernt.

- **Vollständige Partien** liefern die natürliche Verteilung der Stellungen über alle Partiephasen, sind jedoch schief, denn Remis und ruhige Mittelspielstellungen dominieren und klare Taktik ist selten.
- **Taktikaufgaben (Puzzles)** konzentrieren scharfe, forcierte Stellungen, sind aber untypisch und meist auf eine einzige Lösung sowie auf die Gewinnerseite ausgerichtet.
- **Endspieldatenbanken (Tablebases)** liefern exakte, fehlerfreie Bewertungen, allerdings nur für die wenigen Figuren eines Endspiels (bis zu sieben Figuren).

Da unausgewogene Daten das Netz zu Abkürzungen verleiten, etwa stets „Remis“ vorherzusagen, wenn dieses Ergebnis überwiegt, sind Abdeckung und Ausbalancierung von Klassen und Partiephasen ein zentrales methodisches Thema.

Zwei Grenzen bleiben dem Supervised Learning eigen. Erstens die genannte Obergrenze durch die Lehrerquelle und zweitens die Verteilungsverschiebung (Distribution Shift). Im Spiel trifft das Netz auf Stellungen, die aus seinen eigenen Zügen entstehen und im Trainingsdatensatz unterrepräsentiert sein können. Beide Grenzen motivieren Reinforcement Learning durch Self-Play, das seine Daten aus dem eigenen Spiel gewinnt und so die Lehrerquelle hinter sich lassen kann (Abschnitt 2.8).

2.7 Monte Carlo Tree Search (UCT, PUCT)

Die klassische Alpha-Beta-Suche durchsucht den Spielbaum bis zu einer festen Tiefe weitgehend gleichmäßig. Monte Carlo Tree Search (MCTS) verfolgt einen anderen Ansatz: Der Suchbaum wird schrittweise und *asymmetrisch* aufgebaut, indem wiederholt einzelne Pfade von der Wurzel abwärts durchlaufen und die dabei gewonnenen Bewertungen in den besuchten Knoten aggregiert werden [16, 11]. Das Ziel ist hierbei, vielversprechendere Varianten tiefer auszubauen.

Ablauf einer Simulation Jeder Knoten des Baumes entspricht einer Stellung, jede Kante einem Zug. Zu jeder Kante (s, a) werden die Besuchszahl $N(s, a)$, die kumulierte Bewertung $W(s, a)$ und der daraus gemittelte Aktionswert $Q(s, a) = W(s, a)/N(s, a)$ gespeichert. Eine Simulation besteht aus vier Phasen:

1. **Selektion** Von der Wurzel aus wird solange ein Kind nach einer Auswahlregel gewählt, bis ein noch nicht expandierter Knoten erreicht ist. Diese Auswahlregel ist der Kern des Verfahrens und Gegenstand des Folgenden.
2. **Expansion** Der erreichte Knoten wird in den Baum eingefügt und seine legalen Züge als Kanten angelegt.
3. **Simulation oder Evaluation** Die neue Stellung wird bewertet. Klassisch geschieht dies durch eine oder mehrere zufällige Partien bis zum Ende (Rollout), im hier verwendeten Ansatz durch einen einzelnen Forward-Pass des neuronalen Netzes (Abschnitt 2.5).
4. **Backpropagation** Der Bewertungswert wird entlang des durchlaufenen Pfades zur Wurzel zurückgereicht. Dabei werden N und W aktualisiert. Da Bewertungen stets aus Sicht der Seite am Zug gelten, wird das Vorzeichen auf jeder Ebene invertiert.

Nach Erschöpfen des Simulationsbudgets wird an der Wurzel nicht der Zug mit dem höchsten Q gespielt, sondern der meistbesuchte. Besuchszahlen sind robuster, da eine einzelne fehlerhafte Bewertung einen Mittelwert stark verzerren kann, sich in den Besuchen aber nur langsam niederschlägt.

UCT Die Selektion stellt ein Abwägungsproblem zwischen Ausnutzung bereits guter Züge und Erkundung wenig besuchter Alternativen dar. UCT (*Upper Confidence Bounds applied to Trees*) behandelt jeden Knoten als eigenständiges Multi-Armed-Bandit-Problem und überträgt die UCB1-Regel [17] auf den Spielbaum [18]:

$$a^* = \arg \max_a \left[Q(s, a) + c \sqrt{\frac{\ln N(s)}{N(s, a)}} \right], \quad N(s) = \sum_b N(s, b).$$

Der zweite Term wächst, wenn Geschwisterknoten häufig besucht wurden, und schrumpft mit den eigenen Besuchen. Für $N(s, a) = 0$ ist er unbeschränkt. Jeder legale Zug muss also mindestens einmal besucht werden, bevor ein Zug ein zweites Mal ausgewählt wird. Zudem enthält die Regel kein Domänenwissen und alle unbesuchten Züge sind a priori gleichwertig.

Beides ist für Schach ungünstig. Bei einem Verzweigungsgrad von $b \approx 30$ (Abschnitt 2.1) verbraucht bereits die vollständige Erstexpansion einen erheblichen Teil eines realistischen Simulationsbudgets für offensichtlich sinnlose Züge. Hinzu kommt, dass zufällige Rollouts in taktisch scharfen Schachstellungen kaum mit der tatsächlichen Stellungsqualität korrelieren.

PUCT PUCT (*Predictor + UCT*) ersetzt daher den uninformierten Erkundungsterm durch einen, der eine vorab geschätzte Zugwahrscheinlichkeit gewichtet [19]. In der von AlphaZero verwendeten Form [1] lautet die Regel

$$a^* = \arg \max_a [Q(s, a) + U(s, a)], \quad U(s, a) = c_{\text{puct}} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)}.$$

Dabei ist $P(s, a)$ der Prior aus dem Policy-Head, also die über die legalen Züge maskierte und normierte Softmax-Ausgabe (Abschnitt 2.4). Der Prior wirkt multiplikativ. Ein vom Netz als plausibel eingestufter Zug erhält einen um Größenordnungen höheren Erkundungsbonus als ein unplausibler. Damit sinkt der effektive Verzweigungsgrad erheblich, und das Simulationsbudget konzentriert sich auf den relevanten Teil des Baumes.

Der Unterschied im Nenner ($1 + N(s, a)$ statt $\sqrt{N(s, a)}$) ist wichtig. Der Bonus bleibt für unbesuchte Knoten endlich, sodass Züge mit sehr kleinem Prior bei begrenztem Budget nie besucht werden. PUCT gibt die Vollständigkeit von UCT also bewusst zugunsten größerer Suchtiefe auf und wird dadurch von der Qualität des Priors abhängig. Der Wechsel von $\sqrt{\ln N(s)}$ zu $\sqrt{N(s)}$ im Zähler wirkt dem teilweise entgegen, da die Erkundung mit wachsender Simulationszahl langsamer versiegt und die Suche einen systematisch falschen Prior korrigieren kann.

Entscheidend ist, dass die resultierende Besuchsverteilung in der Regel eine bessere Zugsbewertung als der Netz-Prior ist, da sie durch tatsächlich gesuchte Stellungsbewertungen korrigiert wurde. MCTS wirkt damit als Verbesserungsoperator auf die Policy, die Grundlage des in Abschnitt 2.8 beschriebenen Self-Play-Trainings. Implementierungsdetails wie die Behandlung unbesuchter Knoten, die Wahl von c_{puct} und dem Rauschen werden in Abschnitt 3.3 beschrieben.

2.8 AlphaZero-Paradigma: Self-Play Reinforcement Learning

Das AlphaZero-Paradigma verbindet die beiden zuvor beschriebenen Bausteine zu einem geschlossenen Kreislauf [20, 1]: Das Netz steuert über Prior und Bewertung die Suche, und die Suche erzeugt umgekehrt die Daten, mit denen das Netz trainiert wird. Externe Partien oder annotierte Stellungen werden dabei nicht zwingend benötigt. Das Verfahren startet mit zufällig initialisierten Gewichten.

Trainingsziele Grundlage ist die in Abschnitt 2.7 beschriebene Eigenschaft, dass die Besuchsverteilung einer MCTS-Suche eine bessere Zugsbewertung darstellt als der rohe Netz-Prior. Aus jeder im Self-Play gespielten Stellung s_t werden daher zwei Zielsignale gewonnen:

- π_t , die aus den Besuchszahlen abgeleitete Zugverteilung an der Wurzel, als Ziel für den Policy-Head;

- z_t , der tatsächliche Ausgang der Partie aus Sicht der in s_t am Zug befindlichen Seite, als Ziel für den Value-Head.

Der Value-Head lernt also aus dem realen Partieergebnis, der Policy-Head aus der Suche. Trainiert wird auf der gemeinsamen Verlustfunktion

$$\ell = (z - v)^2 - \boldsymbol{\pi}^\top \log \mathbf{p} + c \|\theta\|^2,$$

wobei $\mathbf{p}$ und v die Ausgaben des Netzes bezeichnen und der letzte Term der Regularisierung dient. Bei der hier verwendeten WDL-Ausgabe (Abschnitt 2.5) tritt an die Stelle des quadratischen Fehlers eine Cross-Entropy über die drei möglichen Partieausgänge.

Iterationsschleife Das Training verläuft in Iterationen aus drei Schritten:

1. **Self-Play** Das aktuelle Netz spielt eine feste Anzahl Partien gegen sich selbst. Jede besuchte Stellung wird mit $\boldsymbol{\pi}_t$ und z_t in einen Puffer geschrieben.
2. **Training** Auf Stichproben aus diesem Puffer werden die Gewichte aktualisiert.
3. **Evaluation** Das neue Netz tritt in einem Match gegen die bisherige Iteration an und wird nur dann als neuer Ausgangspunkt übernommen, wenn es einen zuvor festgelegten Punktanteil überschreitet. Andernfalls werden die Gewichte verworfen und die bisherige Version erzeugt weiterhin die Self-Play-Daten.

Dieses Aussieben (*Gating*) verhindert, dass eine verschlechterte Version die Datengenerierung übernimmt. AlphaGo Zero verwendete hierfür eine Schwelle von 55 % über 400 Partien [20]. Der Nachfolger AlphaZero verzichtete auf diesen Schritt und trainierte ein einziges, fortlaufend aktualisiertes Netz [1].

Damit unterscheidet sich dieser Ansatz vom Supervised Learning im Trainingssignal. Statt aus vorgegebenen Zielwerten (Abschnitt 2.6) stammen beide Ziele aus dem System selbst, die Policy aus der eigenen Suche, der Value aus dem eigenen Spielergebnis.

Ein zweiter Unterschied betrifft die Herkunft der Daten. Lernt ein Netz, den Zug eines menschlichen Spielers vorherzusagen, approximiert es dessen Spielweise, einschließlich der darin enthaltenen systematischen Verzerrungen. Etablierte Eröffnungstheorie, positionelle Faustregeln und stilistische Vorlieben sind gewachsene Konventionen, deren Optimalität nicht bewiesen ist. Da sie im gesamten Korpus geteilt werden, mitteln sie sich auch mit wachsender Datenmenge nicht heraus, sondern gehen als systematischer Fehler in das Modell ein. Hinzu kommt eine ungleichmäßige Abdeckung des Stellungsraums. Menschliche Partien konzentrieren sich stark auf bekannte Theorie, sodass gerade jene Stellungen unterrepräsentiert sind, die eine Suche später dennoch betritt.

Self-Play unterliegt diesen Beschränkungen nicht (der Agent sieht allerdings nur, was seine eigene Policy erzeugt, dem wird jedoch durch Dirichlet-Noise am Wurzelknoten entgegengewirkt, Abschnitt 3.3) und erzeugt seine Daten aus genau der Verteilung, in der der Agent anschließend agiert. Dieser Punkt wird von Silver u. a. [20] als wesentlicher Vorteil des Verfahrens angeführt.

Als Beispiel lässt sich der frühe Vorstoß des h-Bauern (h2–h4) anführen. In menschlichen Partien war dieser Zug zum Zeitpunkt der Veröffentlichung von AlphaZero vergleichsweise selten, da der h-Bauer üblicherweise zur Deckung des kurz rochierten Königs benötigt wird. AlphaZero bevorzugte ihn dennoch in einer Vielzahl von Stellungstypen. Offenbar wiegt der damit verbundene Raumgewinn am Königsflügel (und die Möglichkeit, mit h4–h5–h6 den gegnerischen König anzugreifen) in vielen Fällen schwerer als die Schwächung der eigenen Königsstellung.

3 Implementierung

3.1 Modellarchitektur

Beide Agenten, der SL- und der RL-Agent, verwenden dasselbe neuronale Netz, dieselbe Kodierung (Abschnitt 2.4) und zur Zugwahl dieselbe Monte Carlo Tree Search (Abschnitt 2.7, Umsetzung in Abschnitt 3.3). Sie unterscheiden sich einzig im Trainingssignal. Damit misst der Vergleich in Kapitel 4 allein den Effekt des Trainingsparadigmas und nicht den unterschiedlicher Architekturen.

Das Netz folgt dem Residual-Tower-Entwurf von Leela Chess Zero [12, 11] und ist in Abbildung 2 skizziert. Den Eingang bildet der Tensor $8 \times 8 \times 20$ (Abschnitt 2.4), wobei Leela hier 112 Ebenen verwendet, da es die sieben vorangehenden Halbzüge mitstapelt. Eine Initial-Faltung (3×3 , Schrittweite 1) projiziert den Eingang auf 256 Feature Maps, gefolgt von Batch-Normierung und ReLU. Darauf folgt der Tower aus 20 Squeeze-and-Excitation-Residualblöcken und schließlich die beiden Heads. Als Standardkonfigurationen nennt Leela 10×128 , 20×256 und 24×320 (BLOCKS $\times$ FILTERS) [12]. Übernommen wurde die mittlere Variante 20×256 .

Jeder Residualblock enthält zwei 3×3 -Faltungen (Schrittweite 1) mit Batch-Normierung, gefolgt von einer SE-Rekalibrierung, und wird durch eine Skip Connection überbrückt. Die SE-Schicht übernimmt Leelas Aufbau (Abschnitt 2.5). Global Average Pooling verdichtet die 256 Feature Maps zu einem Vektor der Länge FILTERS (256), eine Dense-Schicht reduziert ihn auf SE_CHANNELS (hier 32) mit ReLU, und eine weitere expandiert ihn auf $2 \times \text{FILTERS}$, also je Kanal einen multiplikativen Faktor (Sigmoid) und einen additiven Bias, mit denen die Feature Maps skaliert und verschoben werden.

Der Policy-Head reduziert die Feature Maps über eine 3×3 - und eine 1×1 -Faltung auf die 73 Zugebenen und liest daraus mit einer festen Zuordnung (Gather) die 1858 Zug-Logits aus. Eine Ausgabe-Softmax entfällt, da die Normierung erst nachgelagert bei Loss-Berechnung, Legalitätsmaskierung und Suche erfolgt. Der Value-Head verdichtet das Brett über eine 1×1 - und eine 8×8 -Faltung zu einem Vektor der Länge 128 und gibt über eine Dense-Schicht mit Softmax die WDL-Wahrscheinlichkeiten aus.

Insgesamt besitzt das Netz rund 25 Millionen trainierbare Parameter. Tabelle 2 zeigt ihre Verteilung. Über 96 % entfallen auf den Residual-Tower, während beide Heads zusammen

unter 4 % ausmachen. Innerhalb eines Blocks dominieren die beiden 3×3 -Faltungen, während die SE-Rekalibrierung mit rund 25 000 Parametern nur etwa 2 % je Block beiträgt.

Komponente	Parameter	Anteil
Eingangsblock	46 592	0,2 %
Conv 3×3 (20→256)	46 080	
BatchNorm	512	
SE-Residual-Tower (20 Blöcke)	24 115 840	96,3 %
je Block	1 205 792	
$2 \times$ Conv 3×3 (256→256)	1 179 648	
SE: FC 256 → 32 (ReLU)	8 224	
SE: FC 32 → 256 (Skalierung, Sigmoid)	8 448	
SE: FC 32 → 256 (Bias)	8 448	
$2 \times$ BatchNorm	1 024	
Policy-Head	609 097	2,4 %
Conv 3×3 (256→256)	589 824	
BatchNorm	512	
Conv 1×1 (256→73)	18 761	
Gather → 1858	0	
Value-Head	270 915	1,1 %
Conv 1×1 (256→32)	8 192	
BatchNorm	64	
Conv 8×8 (32→128)	262 272	
Dense (128→3)	387	
Parameter Anzahl	25 042 444	100 %

Tabelle 2: Verteilung der trainierbaren Parameter.

Abweichungen von Leela Gegenüber der ursprünglichen CNN Leela-Architektur wurde das Netz an drei Stellen vereinfacht.

- **Keine Zughistorie** Der Eingabetensor umfasst nur die aktuelle Stellung (20 statt 112 Ebenen). Das verkleinert Netz und Rechenaufwand und macht die Kodierung eindeutig, denn dieselbe Stellung (FEN) wird stets auf denselben Tensor abgebildet. Eine historienbehaftete Kodierung erzeugte je nach Zugreihenfolge unterschiedliche Tensoren (Transpositionen), was die gezielte Untersuchung einzelner Stellungen, etwa bei der Fehlersuche am Value-Head, mehrdeutig machte.
- **Kein Moves-Left-Head** Leela besitzt optional einen dritten Head, der die verbleibende Partielänge schätzt. Er wurde weggelassen, um Parameterzahl und Komplexität zu senken, und ist als optionale Komponente für die Zugwahl verzichtbar.

- **Gather statt Fully-Connected im Policy-Head** Frühere Varianten, sowohl bei Leela [21] als auch in einer früheren Version dieses Projekts, bildeten das 73-Ebenen-Feld über eine voll verbundene Schicht auf die 1858 Züge ab. Der hier verwendete parameterfreie Gather ersetzt diese Schicht. Eine äquivalente Dense-Abbildung von den $73 \times 8 \times 8 = 4672$ Zellen auf 1858 Züge benötigte 8 682 434 Parameter, also etwa ein Drittel des gesamten Netzes.

3.2 Supervised Learning: Datenpipeline, Modell, Training

Das Trainingssignal des SL-Agenten stammt aus einem festen, beschrifteten Korpus, dessen Aufbau dieser Abschnitt von den Rohquellen über die Ausbalancierung und die Stockfish-Annotation bis zum Trainingslauf beschreibt. Der fertige Datensatz umfasst rund 23,5 Mio. Positionen in 471 Chunks zu je 50 000 Beispielen.

Datenquellen und Zielwerte Der Korpus speist sich aus drei Quellen mit unterschiedlichen Stärken (Abschnitt 2.6), zusammengefasst in Tabelle 3.

Spiele Aus etwa 2 TB Lichess-Rohdaten (rund 910 Mio. Partien) verbleiben, nach Filterung auf mindestens 2700 Elo (beide Spieler) und Ausschluss von Bullet-Partien, 324 000 Partien mit bekanntem Ausgang. Jede Stellung einer Partie liefert ein Trainingsbeispiel. Policy-Ziel ist der tatsächlich gespielte (menschliche) Zug als One-Hot-Vektor.

Puzzles Aus der Lichess-Puzzle-Datenbank werden Matt- und Endspielthemen bevorzugt, da menschliches Material diese Situationen durch Aufgabe vor dem Matt nur dünn abdeckt. Die lösende Seite liefert Sieg-Positionen mit dem Lösungszug als Ziel, die verteidigende Seite Niederlage-Positionen mit dem jeweiligen Verteidigungszug.

Tablebase Aus den Syzygy-Endspieldatenbanken (bis fünf Figuren) wird von zufälligen Stellungen aus die tablebase-optimale Linie gespielt und aufgezeichnet, also die gewinnende Seite mit dem schnellsten Fortschritt zum Matt und die verlierende mit dem längsten Widerstand. Ergänzt werden exakt remise Stellungen ohne Policy-Ziel.

Quelle	Policy-Ziel	Value-Label	Anteil
Spiele	gespielter Zug (Mensch)	Soft-Target (SF)	85 %
Puzzles	Lösungs- / Verteidigungszug	Soft-Target (SF)	10 %
Tablebase	DTZ-optimal / – (Remis)	one-hot (exakt)	5 %

Tabelle 3: Die drei Datenquellen mit Policy-Ziel, Value-Label und Anteil je Chunk.

Ausbalancierung von Klassen und Quellen Die Rohquellen sind stark unausgewogen. Menschliche Partien sind remislastig und enden häufig vor dem Matt durch Aufgabe, während Puzzles und Tablebase-Läufe fast ausschließlich entscheidend sind. Ohne Ausgleich verleiten diese Schieflagen das Netz zu den in Abschnitt 2.6 genannten Abkürzungen. Der Datensatz wird daher chunkweise nach einer festen Quote zusammengesetzt. Die Quellen stehen im Verhältnis 85 : 10 : 5 (Spiele : Puzzles : Tablebase), das heißt je 50 000er-Chunk 42 500 Spiel-, 5 000 Puzzle- und 2 500 Tablebase-Positionen. Die drei Value-Klassen Sieg, Remis und Niederlage sind zu je einem Drittel vertreten.

Zwei gezielte Konstruktionen verhindern zusätzlich, dass bereits die Datenquelle den Ausgang verrät.

- Puzzles liefern nicht nur Sieg-, sondern auch Verteidigungspositionen (mit Niederlage-Label), sodass „sieht aus wie ein Puzzle“ nicht länger „Seite am Zug gewinnt“ bedeutet.
- Die Tablebase steuert neben entscheidenden auch exakt remise Stellungen bei, sodass wenige Figuren auf dem Brett nicht pauschal als „entscheidend“ gelernt werden.

Ohne diese Ergänzungen lernte das Netz die Quelle als Abkürzung, statt die Stellung zu bewerten. Die zugehörigen Belege aus den Value-Head-Probes folgen in Abschnitt 4.2.

Stockfish-Annotation (Soft-Targets) Das Value-Ziel für Spiel- und Puzzle-Positionen ist nicht der reine Partieausgang. Dieser ist verrauscht (Abschnitt 2.6), da jede Stellung einer gewonnenen Partie pauschal als Sieg gilt, auch wenn sie selbst ausgeglichen oder unterlegen ist. Jede dieser Positionen wird daher zusätzlich von Stockfish bewertet. Aus einer Suche bis Tiefe 12, einem festen Kompromiss zwischen Bewertungsqualität und dem Aufwand, rund 22,4 Mio. Positionen zu annotieren, liefert Stockfish eine WDL-Verteilung q . Mit dem als One-Hot kodierten Partieausgang z ergibt sich das Soft-Target als konvexe Mischung

$$t = \lambda z + (1 - \lambda) q, \quad \lambda = 0,5.$$

Der Wert $\lambda = 0,5$ gewichtet Partieausgang und Stellungsbewertung gleich und entspricht damit der gleichgewichtigen Mischung aus Ausgang und Suchbewertung. Tablebase-Positionen bleiben one-hot, da die Syzygy-Bewertung exakt ist und keiner Glättung bedarf.

Chunking und Training Der ausbalancierte Datensatz wird in 471 Chunks zu je 50 000 Positionen geschrieben (Format `npz` mit Brett-Tensor, Policy- und Value-Ziel). Zwei Chunks (100 000 Positionen) werden als Validierungsmenge zurückgehalten. Trainiert wird dasselbe Netz wie auf der RL-Seite (Abschnitt 3.1). Der Gesamt-Loss summiert Policy- und Value-Loss. Da der Policy-Head rohe Logits ausgibt (Abschnitt 3.1), ist der Policy-Loss eine Cross-Entropy über die Logits, der Value-Loss eine Cross-Entropy über die drei WDL-Klassen. Remise Tablebase-Positionen besitzen kein Policy-Ziel und werden

über ein Nullgewicht aus dem Policy-Loss ausgeschlossen. Optimierer, die stufenweise gesenkte Lernrate und die Regularisierung folgen AlphaZero [1] und sind in Tabelle 4 zusammengefasst. Der Rechenaufwand und die daraus folgende Beschränkung auf eine Epoche werden in Abschnitt 4.5 behandelt.

Hyperparameter	Wert
Optimierer	AdamW (Weight Decay 10^{-4} , nur Kernel)
Lernrate	$10^{-4} \rightarrow 10^{-5}$ (70 %) $\rightarrow 10^{-6}$ (90 %)
Batch-Größe	512
Epochen	1 (über 469 Chunks, jeweils 50 000 Positionen)
Policy-Loss	Cross-Entropy (aus Logits)
Value-Loss	Cross-Entropy (WDL)

Tabelle 4: Trainingskonfiguration des SL-Trainings.

Während des Trainings werden regelmäßig Checkpoints gesichert. Sie bilden die Grundlage für den Vergleich der Trainingsstände und der verschiedenen SL-Varianten in Abschnitt 4.2.

3.3 Reinforcement Learning: MCTS-Kern, Self-Play, Trainings-Loop

Der RL-Agent nutzt dasselbe Netz (Abschnitt 3.1) und dieselbe Kodierung (Abschnitt 2.4) wie der SL-Agent und unterscheidet sich allein darin, dass er sein Trainingssignal im Self-Play selbst erzeugt (Abschnitt 2.8). Der Kreislauf aus Suche, Datenerzeugung und Netz-Update folgt dem AlphaZero-Verfahren [1] in der didaktisch reduzierten Form von Klein [11]. Die beiden zentralen Stellhebel, die Zahl der Simulationen pro Zug und die Zahl der Self-Play-Games pro Iteration, werden empirisch in Abschnitt 4.3 kalibriert.

MCTS-Kern Die Zugwahl im Self-Play nutzt die PUCT-Suche aus Abschnitt 2.7. Noch nicht besuchte Children-Nodes erhalten über eine First-Play-Urgency-Absenkung einen leicht reduzierten Startwert, sodass die Suche den vom Policy-Head bevorzugten Zügen folgt, statt jeden legalen Zug einmal erzwingen zu müssen. Am Root-Node mischt Dirichlet-Noise mit $\alpha = 0,3$ und $\varepsilon = 0,25$ zusätzliche Exploration bei (Abschnitt 2.8), die nur im Self-Play aktiv ist und bei der Stärkemessung (Abschnitt 4.1) abgeschaltet wird.

Self-Play Die Trainingspartien erzeugt jeweils das aktuell beste Netz. In den ersten 30 Halbzügen werden die Züge proportional zu den Besuchszahlen der Children-Nodes gesampelt (Temperatur 1), um die Eröffnungen zu variieren, danach spielt das Netz greedy den meistbesuchten Zug (Abschnitt 2.8). Für jede besuchte Stellung entstehen zwei Trainingsziele, die Besucherverteilung an der Wurzel als Policy-Ziel und der Partieausgang

als Value-Ziel. Diese Samples wandern über ein *sliding window* der letzten Iterationen in einen Replay Buffer, aus dem der Trainingsschritt zieht.

Gated Training-Loop Jede Iteration durchläuft drei Schritte. Zuerst erzeugt das beste Netz eine feste Zahl Self-Play-Games. Danach wird ausgehend vom besten Netz ein Kandidat trainiert. Zuletzt tritt der Kandidat in einem Match gegen das beste Netz an, unter festen Eröffnungen und ohne Noise wie bei der Stärkemessung (Abschnitt 4.1), und ersetzt es nur bei Überschreiten einer festen Punktschwelle. Dieses Gating verhindert, dass eine verschlechterte Version die Datenerzeugung übernimmt, ein Schritt, den das ursprüngliche AlphaZero weglässt [1], der sich hier aber unter knappem Rechenbudget als notwendig erwies (Abschnitt 4.3).

4 Evaluation & Ergebnisse

4.1 Versuchsaufbau

Damit die Ergebnisse beider Agenten auf einer gemeinsamen Skala vergleichbar bleiben, folgen alle Spielstärkemessungen dieses Kapitels demselben Messverfahren. Die ansatzspezifischen Details, also welche Varianten gegen welche Gegner antreten, benennen die jeweiligen Ergebnisabschnitte.

Spielprotokoll Eine Messung besteht aus einem farbausgeglichene Match zwischen zwei Gegnern. Als Eröffnungen dienen zufällig erzeugte Sequenzen von je drei Halbzügen. Jede Eröffnung wird zweimal gespielt, einmal mit vertauschten Farben. Der Farbtasch hebt den Vorteil des Anziehenden auf, und die zufälligen Eröffnungen verhindern, dass zwei deterministisch spielende Netze wiederholt dieselbe Partie austragen. Ein Netz wählt seine Züge über MCTS (Abschnitt 2.7) mit 1000 Simulationen pro Zug. Partien, die nach 400 Halbzügen nicht entschieden sind, werden als Remis gewertet. Der Umfang eines Matches ist bei den jeweiligen Ergebnissen angegeben. Als Referenzgegner und für alle Stellungsbewertungen dient Stockfish 17.1.

Wahl der Suchparameter Die Einstellungen der Suche wurden vorab an 393 von Stockfish annotierten Teststellungen (Tiefe 16) über ein Gitter aus c_{puct} , First-Play-Urgency-Absenkung und Simulationszahl geprüft, gemessen am mittleren Centipawn-Verlust der gewählten Züge. Das Ergebnis ist weitgehend ein Null-Resultat. c_{puct} bildet zwischen 1,5 und 4,0 ein flaches Plateau (nur 0,75 fällt signifikant ab), und die First-Play-Urgency-Absenkung zeigt im untersuchten Bereich keinen messbaren Effekt. Allein die Simulationszahl wirkt durchgehend, denn der Centipawn-Verlust fällt monoton mit ihr. Deshalb wurde $c_{\text{puct}} = 1,5$ beibehalten und die Simulationszahl mit 1000 so hoch gewählt, wie es das Zeitbudget der Matches zulässt.

Elo-Anker Aus dem in einem Match erzielten Punktanteil S einer Seite folgt nach der in Abschnitt 2.2 eingeführten Umkehrung $\Delta R = 400 \cdot \log_{10}(S/(1 - S))$ die geschätzte Elo-Differenz zum Gegner, begleitet von einem 95 %-Konfidenzintervall aus der Match-Stichprobe. Da das Elo-System nur Differenzen liefert, dient als absoluter Anker Stockfish mit über UCI_Elo gedrosselter Spielstärke. Konkret tritt das zu verankernde Netz gegen zehn Stockfish-Stufen von 1400 bis 2300 Elo (Schrittweite 100) an, bei einer festen Bedenkzeit von 0,75s je Zug auf Stockfish-Seite. Das Netz zieht wie oben über die MCTS-Suche mit 1000 Simulationen. Je Stufe werden zwanzig Partien gespielt, wieder farbausgeglichen über zufällige Eröffnungen. Aus dem Punktverlauf über die Stufen wird die absolute Elo des Netzes per logistischer Anpassung der Erwartungsscore-Kurve geschätzt.

4.2 Ergebnisse Supervised Learning

Der SL-Agent entstand in vier Iterationen, die sich allein in der Zusammensetzung und Beschriftung der Trainingsdaten unterscheiden (Abschnitt 3.2). Dasselbe Netz wurde also viermal auf jeweils verbessertem Datenmaterial trainiert, und jede Iteration war durch das Versagen der vorangegangenen motiviert. Zwei Betrachtungsweisen ziehen sich durch die Auswertung. Gezielte *Probes* des Value-Heads prüfen, wie gut das Netz eine Stellung *bewertet*, während Matches prüfen, wie gut es *spielt*. Die Probes greifen auf Stellungen aus den Syzygy-Endspieldatenbanken zurück, deren Ausgang exakt bekannt ist, sodass sich eine Fehlbewertung zweifelsfrei als solche ausweisen lässt.

v1, nur Partien Die erste Variante lernte ausschließlich aus Partien von Spielern über 2700 Elo. Da rund 60 % dieser Partien remis enden, kollabierte der Value-Head auf die Remis-Vorhersage. Zudem fehlte dem Netz jedes Ziel im Endspiel, da menschliche Partien meist vor dem Matt durch Aufgabe enden. Diese Variante konnte praktisch nicht mattsetzen und diente lediglich als Ausgangspunkt der folgenden Ausbalancierung.

v2, Ausbalancierung Um beide Schwächen zu beheben, wurde der Datensatz nach Quelle (85 : 10 : 5 für Spiele : Puzzles : Tablebase) und Ausgang (je ein Drittel Sieg, Remis, Niederlage) ausbalanciert (Abschnitt 3.2). Tabelle 5 zeigt die Zusammensetzung eines Chunks. Das Netz konnte nun mattsetzen, lernte aber eine neue Abkürzung. In v2 sind *alle* Puzzle-Positionen Siege und die Tablebase kennt keine Remisen, sodass die Quelle einer Stellung ihren Ausgang verrät. Statt die Stellung zu bewerten, lernte das Netz „Puzzle bzw. wenige Figuren $\Rightarrow$ entschieden“ und verallgemeinerte dies zu „Materialungleichgewicht $\Rightarrow$ entschieden“, unabhängig davon, wem das Material fehlt. Die Probes belegen dies. Auf beweisbar remisenden Stellungen mit Materialungleichgewicht sagt v2 im Mittel nur $P(\text{Remis}) = 0,385$ voraus, und auf beweisbar verlorenen Verteidiger-Stellungen erkennt es nur 27,7 % korrekt als Niederlage.

Quelle	Var.	Sieg	Remis	Niederlage	Summe
Spiele	v2	10 417	16 667	15 416	42 500
	v3	13 334	15 834	13 332	42 500
Puzzles	v2	5 000	0	0	5 000
	v3	2 500	0	2 500	5 000
Tablebase	v2	1 250	0	1 250	2 500
	v3	833	833	834	2 500
Summe	v2	16 667	16 667	16 666	50 000
	v3	16 667	16 667	16 666	50 000

Tabelle 5: Zusammensetzung eines 50 000er-Chunks nach Quelle und Ausgang.

Beide Varianten halten dieselben Quellenanteile (85 : 10 : 5) und je ein Drittel je Ausgang. Sie unterscheiden sich nur darin, *wie* sich die Ausgänge auf die Quellen verteilen. v4 übernimmt die Verteilung von v3 unverändert und ändert lediglich die Value-Labels von Spiel- und Puzzle-Positionen (Soft-Target statt one-hot).

v3, Anti-Abkürzung v3 verwendet dieselben Quellenanteile, ergänzt aber gezielt Gegenbeispiele. Puzzles liefern nun auch Verteidiger-Positionen (mit Niederlage-Label) und die Tablebase auch exakt remise Stellungen (Tabelle 5). Damit ist die Quelle nicht länger mit dem Ausgang korreliert, und die Abkürzung verschwindet. Die korrekte Niederlage-Erkennung der Verteidiger steigt von 27,7 % auf 98,1 %, und $P(\text{Remis})$ auf den Materialungleichgewichts-Remisen von 0,385 auf 0,781. Auch die Kalibrierung auf echten Partie-Positionen verbessert sich leicht. Es blieb jedoch ein etwas verrauschter Value-Head mit taktischer Blindheit im Mittelspiel.

v4, Stockfish-Soft-Targets Die letzte Variante ändert nur die Value-Labels. Spiel- und Puzzle-Positionen erhalten statt des reinen Partieausgangs ein mit Stockfish (Tiefe 12) gemischtes Soft-Target (Abschnitt 3.2), während die Tablebase-Labels exakt bleiben. Das Ergebnis ist zweiseitig, denn v4 ist der bessere *Bewerter*, aber der schlechtere *Ausgangs-Prädiktor*. Der Abgleich mit Stockfish verbessert sich deutlich (mittlerer absoluter Fehler von 0,120 auf 0,097), während die Kalibrierung auf den tatsächlichen Partieausgang zurückgeht (Cross-Entropy des Value-Heads von 0,755 auf 0,831). Das ist zu erwarten, da v4 gerade auf die Stockfish-Bewertung hin optimiert wurde, die den Wert einer einzelnen Stellung genauer, aber anders als das grobe Partieergebnis beschreibt.

Direkter Vergleich v3 gegen v4 Ob die bessere Bewertung auch stärkeres Spiel bedeutet, entscheidet ein direktes Match (Abschnitt 4.1). Ohne Suche, allein anhand des Policy-Arguments des Netzes, sind beide Varianten praktisch gleich stark (Punktanteil

0,49 für v3, innerhalb der statistischen Unsicherheit). Unter der vollständigen Suche mit 1000 Simulationen jedoch gewinnt v4 klar. Über 100 Partien erreicht v4 einen Punktanteil von 0,645 (54 – 21 – 25), entsprechend einer Differenz von rund +104 Elo (95 %-Konfidenzintervall des gegnerischen Punktanteils 0,272 – 0,438, also signifikant). Der verbesserte Value-Head schlägt sich demnach erst in Kombination mit der Suche in Spielstärke nieder. Die Suche stützt sich auf die Stellungsbewertung, und eine genauere Bewertung führt zu besseren Zügen.

Konvergenz Abbildung 3 zeigt den Trainingsverlauf von v4 über die eine Epoche. Der Gesamt-Loss wird vom Policy-Loss dominiert (Kreuzentropie über 1858 Züge), während der Value-Loss (über drei Klassen) klein und früh stabil ist. Training und Validierung laufen ohne Überanpassung nahe beieinander. Die Aufschlüsselung nach Datenquelle zeigt, wo das Netz unterschiedlich gut abschneidet. Der Value-Head bewertet Tablebase-Stellungen nahezu fehlerfrei (Accuracy 99 %), Puzzles gut (81 %) und die mehrdeutigen Spiel-Positionen am schwächsten (60 %). Der Policy-Head sagt forcierte Puzzle-Züge (76 %) besser voraus als menschliche Spiel-Züge (50 %), deren Vorhersage naturgemäß mehrdeutig ist.

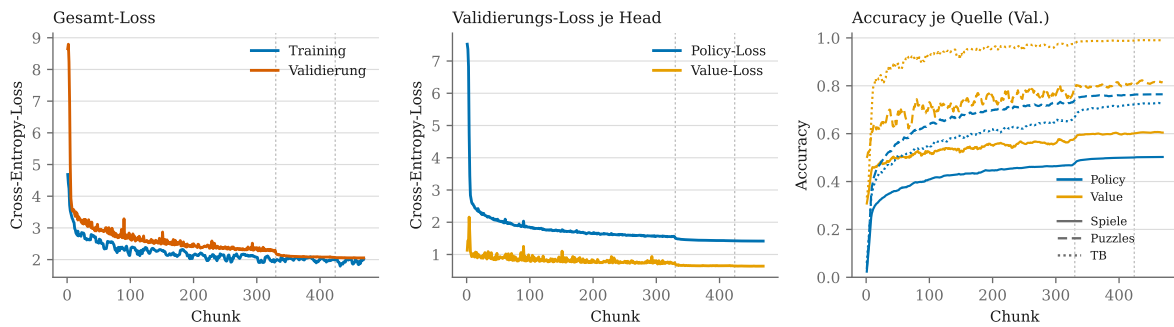


Abbildung 3: Trainingsverlauf von v4 über die 469 Chunks der einen Epoche.

Links steht der Gesamt-Loss (Training gegen Validierung), in der Mitte dessen Zerlegung in Policy- und Value-Loss auf den Validierungsdaten, rechts die Validierungs-Accuracy je Datenquelle, wobei die Farbe den Head und der Linienstil die Quelle bezeichnet. Die gestrichelten Vertikalen markieren die Absenkung der Lernrate bei 70 % und 90 % des Trainings.

Dass der fallende Loss tatsächlich mit gewonnener Spielstärke einhergeht, prüft ein Vergleich des fertigen Netzes gegen 30 eigene Trainings-Checkpoints (Abbildung 4), je 50 Partien. Um allein das Gelernte zu messen, spielen beide Seiten dabei ohne Suche, also direkt das Argument des Policy-Heads. Die Spielstärke steigt über das Training deutlich an, denn frühe Checkpoints liegen rund 500 Elo zurück. Gegen Ende sättigt sie jedoch, und ab etwa Chunk 400 sind die Checkpoints vom finalen Netz nicht mehr sicher zu unterscheiden. Das letzte Sechstel des Trainings trug damit kaum noch messbare Spielstärke bei.

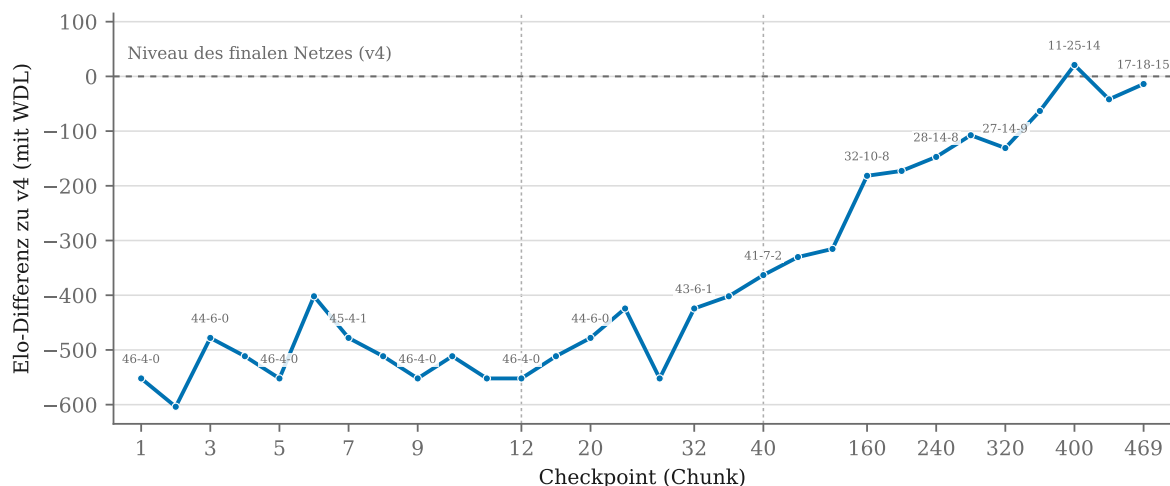


Abbildung 4: Spielstärke der Trainings-Checkpoints von v4 relativ zum fertigen Netz.

An den beschrifteten Punkten ist zusätzlich die Bilanz Sieg-Remis-Niederlage aus Sicht des fertigen Netzes angegeben. Die Checkpoints sind gleichmäßig angeordnet und nicht proportional zur Trainingsdauer, da der frühe Verlauf dichter abgetastet wurde. Die gestrichelten Vertikalen markieren, wo der Abstand zwischen den gesicherten Checkpoints wächst (ab Chunk 12 jeder vierte, ab Chunk 40 jeder vierzigste).

Stärke gegenüber den Vorvarianten Dass die Datenrezept-Iterationen nicht nur die Bewertung, sondern auch das Spiel verbesserten, bestätigen direkte Matches gegen die früheren Netze. In kürzeren 30-Partie-Matches unter demselben Suchbudget schlägt v4 die Ausbalancierungs-Variante (v2) mit einem Punktanteil von 0,93 (26-4-0) und das reine Partie-Netz (v1) mit 0,90 (24-6-0), in beiden Fällen ohne eine einzige Niederlage. Die schrittweise verbesserte Datenaufbereitung übersetzt sich damit durchgängig in Spielstärke.

Absolute Spielstärke Zur Einordnung auf eine absolute Skala tritt v4 gegen die zehn Stockfish-Stufen des Ankers an (Abschnitt 4.1). Tabelle 6 zeigt Bilanz und Punktverlauf über je zwanzig Partien. Der Übergang zum ausgeglichenen Match (Punktanteil 0,5) liegt bei etwa 1900 Elo. Eine logistische Anpassung der Erwartungsscore-Kurve an alle zehn Stufen schätzt die absolute Spielstärke von v4 auf rund 1920 Elo (95 %-Likelihood-Intervall etwa 1855-1980). Der Punktverlauf ist über die zwanzig Partien je Stufe weitgehend monoton, und die verbleibende Reststreuung gegen die stärksten Stufen fällt kaum noch ins Gewicht, sodass der Anker eine belastbare Schätzung liefert. Zur Veranschaulichung dieser Größenordnung lässt sich der Anker auf die wöchentliche Blitz-Ratingverteilung von Lichess beziehen [22]. Rund 1920 Elo liegt dort oberhalb von etwa 80 % der aktiven Spieler. Der Vergleich ist allerdings nicht exakt, da die hier verwendete

Stockfish-verankerte Elo-Näherung nicht auf derselben Skala wie das Glicko-2-System von Lichess liegt.

Stockfish-Elo	1400	1500	1600	1700	1800	1900	2000	2100	2200	2300
Siege	20	17	18	11	13	8	2	0	1	1
Remisen	0	3	1	5	1	3	9	7	5	8
Niederlagen	0	0	1	4	6	9	9	13	14	11
Punktanteil v4	1,00	0,93	0,93	0,68	0,68	0,48	0,33	0,18	0,18	0,25

Tabelle 6: Bilanz und Punktanteil von v4 gegen die zehn Stockfish-Anker-Stufen.

Spielverlauf nach Phasen Wo genau v4 im Spielverlauf Boden verliert, zeigt eine phasenaufgelöste Analyse der Partien gegen die beiden Stufen um die Ankerstärke des Netzes, nämlich Stockfish 1900 (knapp darunter, Punktanteil 0,48) und Stockfish 2000 (knapp darüber, Punktanteil 0,33). Jede Stellung aller je zwanzig Partien wurde dazu von Stockfish bei voller Stärke (Tiefe 16) bewertet. Die Centipawn-Bewertung c einer Stellung (aus Sicht des Anziehenden) wurde anschließend über die von Lichess an eigenen Partien angepasste logistische Funktion [23] in einen Gewinnvorteil überführt,

$$V(c) = 100 \cdot \left(\frac{1}{1 + e^{-0,00368c}} - \frac{1}{2} \right) \quad [\text{Gewinn-\%Punkte}],$$

sodass große Materialvorteile nicht beliebig stark ins Gewicht fallen. Abbildung 5 mittelt diesen, aus Sicht von v4 orientierten, Vorteil (positiv für v4, negativ für Stockfish) je Zugnummer über alle Partien. Beide Kurven erzählen dieselbe Geschichte. v4 startet ausgeglichen bis leicht besser und verliert erst im weiteren Verlauf die Kontrolle, gegen die stärkere Stufe 2000 jedoch deutlich früher und tiefer. Gegen Stockfish 1900 hält v4 bis weit ins Mittelspiel einen Vorteil (Mittelwert +3,9 in der Eröffnung, +11,7 im Mittelspiel) und gibt erst spät nach (+7,0 im Endspiel). Gegen Stockfish 2000 bleibt es früh nur ausgeglichen (+1,2 bzw. +0,7) und wird schon ab etwa Zug 23 klar überspielt (−22,3 im Endspiel). Die Schwäche des SL-Netzes liegt somit nicht in der Eröffnungs- oder Mittelspielbehandlung, sondern in der Verwertung und Endspieltechnik, konsistent damit, dass menschliche Partien selten bis zum Matt gespielt werden und der Datensatz das Endspiel daher nur dünn abdeckt. Gegen den stärkeren Gegner, der Ungenauigkeiten konsequenter bestraft, tritt dieser Bruch entsprechend früher zutage.

Nach oben ist v4 im Vorteil, nach unten Stockfish. Die vertikale Achse ist für beide Stufen gleich skaliert. Die Kurven enden dort, wo weniger als drei Partien beitragen.

4.3 Ergebnisse Reinforcement Learning

Dieser Abschnitt verfolgt die Entwicklung des mit RL trainierten Agenten über mehrere Self-Play-Läufe und leitet daraus die Konfiguration ab, die unter einem studentischen

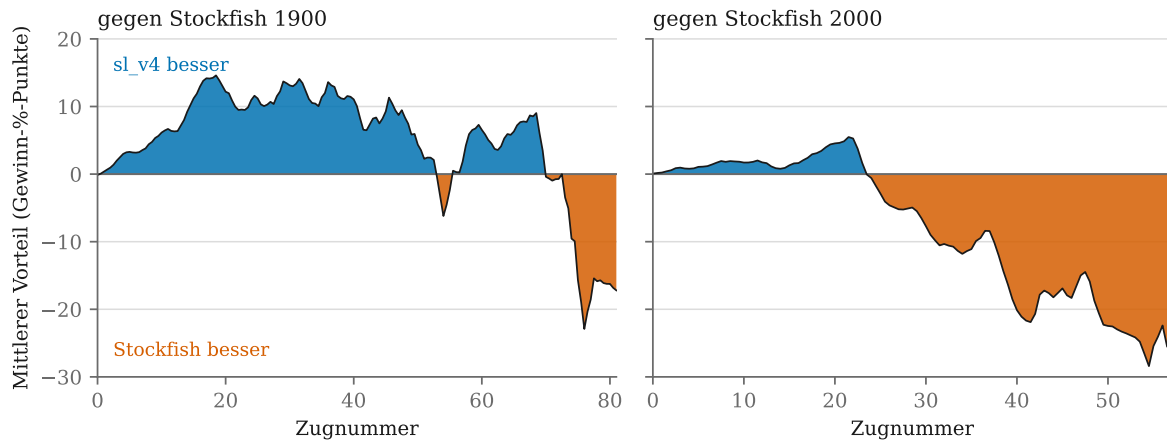


Abbildung 5: Mittlerer Spielverlauf von v4 gegen Stockfish 1900 (links) und Stockfish 2000 (rechts) über je zwanzig Partien.

Rechenbudget zu konsequentem Lernen führte. Die Messungen sind hier intern, Candidate gegen das vorige beste Netz oder Checkpoint gegen Checkpoint, und dienen dem Vergleich der RL-Konfigurationen untereinander. Ihre Einstellungen weichen teils vom Protokoll in Abschnitt 4.1 ab und werden je Lauf benannt. Die Einordnung auf die absolute Stockfish-Skala und der direkte Vergleich mit dem SL-Agenten folgen in Abschnitt 4.4.

Warum nicht from scratch? Ein Agent von zufälligen Gewichten aus zu trainieren erwies sich unter dem verfügbaren Budget als aussichtslos. Nach rund 24 h Self-Play holte das Netz gegen einen Random-Mover nur 26 von 50 Punkten. Ein Netz, das gegen zufällige Züge kaum über Zufallsniveau kommt, stellt im Mittel etwa so häufig Figuren ein wie der Gegner selbst. Daraus folgt die Grundentscheidung des RL-Teils, nicht *from scratch* zu lernen, sondern ein per Supervised Learning vortrainiertes Netz als Seed zu verwenden und dieses per Self-Play zu verbessern.

Erster Bootstrap-Versuch und sein Scheitern Der erste Lauf startete aus einem vollständig konvergierten SL-Netz mit 100 Simulationen pro Zug und 10 Self-Play-Games je Iteration, ohne Gating, und übernahm jedes trainierte Netz bedingungslos. Über rund 30 h entstanden knapp 400 Iterationen, doch das Netz wurde nicht stärker, sondern schwächer. In einem Turnier zwischen den Checkpoints gewann das SL-Startnetz alle 24 Direktpartien gegen sämtliche RL-Checkpoints, und die späteren Checkpoints schlugen die früheren nicht konsequent. Die Loss-Kurven benennen die Ursache. Der Policy-Loss blieb über den gesamten Lauf flach bis leicht steigend, während der Value-Loss fiel. Ein flacher Policy-Loss bedeutet, dass die Suche keine besseren Zielverteilungen liefert als das Netz selbst, es also nichts zu lernen gibt. Der fallende Value-Loss ist irreführend, da er nur zeigt, dass das Netz die Ausgänge seines eigenen, remislastigen Self-Plays gut vorhersagt. Bei nur 100 Simulationen ist MCTS(Netz) (diese Schreibweise wird im Folgenden genutzt

für die Anreicherung eines Netzes mit MCTS) kaum stärker als das rohe Netz, sodass das Trainingsziel näherungsweise dem eigenen Zug des Netzes entspricht und das Update lediglich Rauschen in eine bereits starke Policy schiebt.

Die bindende Bedingung Damit im Self-Play etwas zu lernen ist, muss MCTS(Netz) stärker sein als das Netz allein. Diese Bedingung stellt zwei Anforderungen, die beide erfüllt sein müssen, und die folgenden Läufe zeigen, dass das Verletzen je einer Anforderung bereits genügt, um das Lernen zu verhindern. Ein Lauf mit starkem SL-Seed, 800 Simulationen und weiterhin nur 10 Spiele je Iteration, nun mit Gate, akzeptierte über zehn Iterationen keinen einzigen Candidate. Die Candidates verloren durchgehend gegen das beste Netz, das Gate fing die Verschlechterung also korrekt ab. Zehn Spiele liefern zu wenige und zu selbstähnliche Positionen, um eine konvergierte Policy in eine bessere Richtung zu schieben. Ein weiterer Lauf prüfte den umgekehrten Fall mit sehr vielen Spielen bei schwacher Suche, 2000 Spiele je Iteration bei nur 100 Simulationen. Über drei Iterationen und rund 15 h lernte das Netz nicht, es holte gegen seinen Stand von 15 h zuvor nur 10,5 von 20 Punkten. Eine hohe Zahl an Spielen kompensiert eine zu schwache Suche also nicht. Zugleich verdeutlicht dieser Lauf die Kosten, denn bei rund 5 h je Iteration verbrauchten schon drei Iterationen das halbe wöchentliche Kaggle-Kontingent.

Die funktionierende Konfiguration Erst die Kombination aus ausreichend starker Suche und ausreichend vielen Spielen führte zu konsequentem Lernen. In der Konfiguration mit 800 Simulationen und 100 Spiele je Iteration, aus einem bewusst schwächer gewählten Seed und mit Gate, wurden vier von sechs Candidates promotet, die Kette klettert also mehrfach in Folge nach oben. Anders als die zuvor genannten Läufe erzeugt diese Konfiguration ein echtes Aufwärtssignal. Die vier nacheinander promoteten Netze bezeichnen wir als rl_1 bis rl_4, ihren Ausgangspunkt (den SL-Checkpoint sl_160) als rl_0. Das finale Netz rl_4 vertritt im Folgenden den RL-Agenten. Einschränkend gilt, dass hier zwei Faktoren zusammenwirken, die höhere Game-Zahl und der schwächere Seed, der der Suche mehr Spielraum über dem Netz lässt. Der nachgewiesene Fortschritt ist damit ein Aufstieg von einer schwachen Basis. Dass das trainierte Netz auch das starke SL-Netz übertrifft, ist dadurch nicht belegt und wird in Abschnitt 4.4 geprüft.

Einordnung unter studentischem Budget AlphaZero und Leela erzeugen je Netzgeneration Zehntausende bis Millionen Partien, Klein nennt für AlphaGo Zero rund 25 000 Spiele je Iteration [11]. Bei rund 5 h für 2000 Spiele mit reduzierter Suche liegt diese Größenordnung jenseits des Erreichbaren. Der empirisch beste Weg war daher nicht zu versuchen, die Game-Menge von AlphaZero nachzuahmen, sondern das Gegenteil, eine moderate Zahl von Spielen bei hinreichend starker Suche über möglichst viele gegatete Iterationen. Der detaillierte Rechenaufwand und die Inferenzzeiten folgen in Abschnitt 4.5.

Absolute Einordnung gegen Stockfish Zur Einordnung auf eine absolute Skala tritt rl_4 wie der SL-Agent gegen die zehn Stockfish-Stufen des Ankers an (Abschnitt 4.1), je zwanzig Partien pro Stufe. Tabelle 7 zeigt den Punktverlauf. Schon gegen die schwächste Stufe von 1400 Elo bleibt rl_4 mit einem Punktanteil von 0,38 unter dem ausgeglichenen Match und fällt über die stärkeren Stufen rasch auf Werte um 0,05. Der Übergang zum ausgeglichenen Match liegt damit unterhalb der schwächsten über UCI_Elo einstellbaren Stufe, sodass die absolute Spielstärke nur per Extrapolation zu schätzen ist. Abbildung 6 zeigt ergänzend, dass rl_4 bereits in der Eröffnungsphase in Rückstand gerät, ein Muster, das die Herkunft der reinen Self-Play-Daten ohne Eröffnungswissen widerspiegelt. Die Gegenüberstellung mit dem entgegengesetzten Schwächeprofil des SL-Agenten folgt in Abschnitt 4.4.

Stockfish-Elo	1400	1500	1600	1700	1800	1900	2000	2100	2200	2300
Siege	7	5	2	0	0	1	0	0	0	0
Remisen	1	3	2	3	3	1	1	2	2	2
Niederlagen	12	12	16	17	17	18	19	18	18	18
Punktanteil rl_4	0,38	0,33	0,15	0,08	0,08	0,08	0,03	0,05	0,05	0,05

Tabelle 7: Bilanz und Punktanteil von rl_4 gegen die zehn Stockfish-Anker-Stufen.

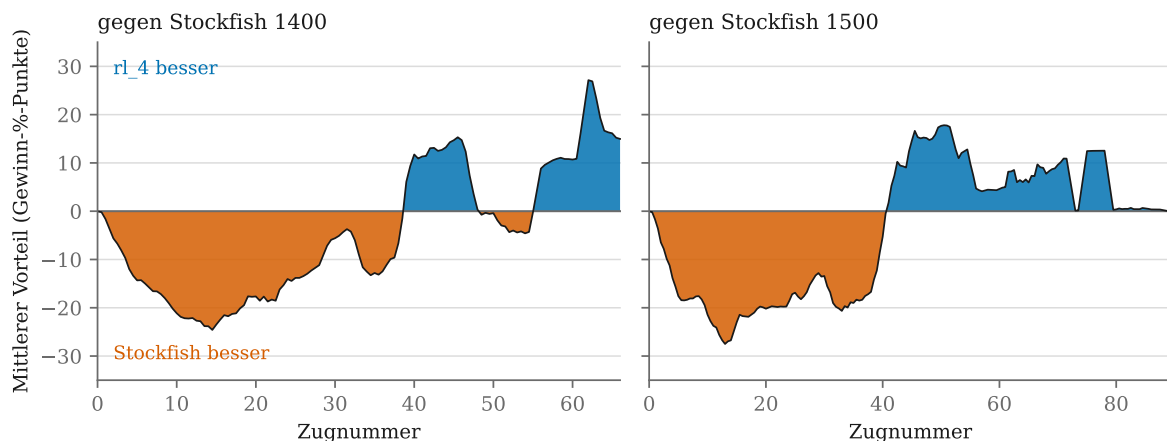


Abbildung 6: Mittlerer Spielverlauf von rl_4 gegen Stockfish 1400 (links) und Stockfish 1500 (rechts) über je zwanzig Partien.

4.4 Direktvergleich: SL vs. RL

Der abschließende Vergleich stellt die besten Netze beider Ansätze gegenüber, also das per Supervised Learning trainierte v4 (Abschnitt 4.2) und das im Self-Play trainierte rl_4 (Abschnitt 4.3).

Direktes Match Unter dem Protokoll aus Abschnitt 4.1 erreicht rl_4 über 50 Partien mit je 1000 Simulationen gegen v4 einen Punktanteil von 0,07 (1–5–44), was einer Differenz von rund –449 Elo entspricht.

Einordnung über den Anker Beide Netze wurden unabhängig voneinander gegen dieselben zehn Stockfish-Stufen vermessen (Tabellen 6 und 7). Für v4 ergibt die logistische Anpassung rund 1920 Elo. Für rl_4 liefert dieselbe Anpassung über alle Stufen rund 1360 Elo, über die aussagekräftigen unteren Stufen jedoch nur rund 1330 Elo. Der niedrigere Wert ist der belastbarere, denn gegen die stärksten Stufen stammen nahezu alle von rl_4 erzielten Punkte aus Remisen durch Stellungswiederholung, was die Anpassung nach oben zieht. Unabhängig davon bleibt die Zahl eine Extrapolation, da rl_4 selbst gegen die schwächste Stufe nur einen Punktanteil von 0,38 erreicht und der Übergang zum ausgeglichenen Match damit unterhalb des über UCI_Elo erreichbaren Bereichs liegt. Der so geschätzte Abstand von rund 590 Elo weist in dieselbe Richtung wie das direkte Match und liegt in derselben Größenordnung.

Self-Play verbesserte seinen Ausgangspunkt Der Rückstand bedeutet nicht, dass das Self-Play wirkungslos blieb. Gegen jenes Netz, aus dem es gestartet wurde, gewinnt rl_4 deutlich. Über je 100 Partien erreicht es gegen den Seed-Checkpoint sl_160 einen Punktanteil von 0,77 (69–15–16, rund +205 Elo) und gegen den nächsten gesicherten Checkpoint sl_200 einen von 0,70 (61–17–22, rund +143 Elo). Beide Matches ordnen die Checkpoints zugleich in der erwarteten Reihenfolge, denn sl_200 erweist sich als das stärkere der beiden. Das Self-Play hat seinen Ausgangspunkt also um gut 150 bis 200 Elo verbessert. Der Abstand zu v4 entsteht demnach nicht daraus, dass Self-Play unter diesen Bedingungen nichts lernt, sondern daraus, dass es von einem deutlich schwächeren Netz aus startete und diesen Vorsprung im verfügbaren Budget nicht aufholen konnte.

Unterschiedliche Schwächen Aufschlussreicher als die reine Differenz ist, wo die beiden Agenten Boden verlieren. Die phasenaufgelösten Verläufe (Abbildungen 5 und 6) zeigen entgegengesetzte Muster. v4 hält Eröffnung und Mittelspiel ausgeglichen und wird erst im Endspiel überspielt, während rl_4 bereits nach zwölf Zügen im Mittel 14 bis 16 Gewinn-%-Punkte zurückliegt und den größten Teil seines Rückstands lange vor dem Endspiel einbüßt. Dieses Muster spiegelt unmittelbar die Herkunft der Trainingsdaten. Menschliche Partien decken Eröffnung und Mittelspiel dicht ab und enden meist vor dem Matt, während Self-Play seine Partien stets zu Ende spielt, aber ohne jedes Eröffnungswissen beginnt.

4.5 Rechenaufwand & Inferenzzeit

Datenaufbereitung und Training (SL) Der teuerste Posten war die Datengrundlage. Der Lichess-Partiendump umfasst rund 2 TB, dessen Filterung etwa 13 Stunden in

Anspruch nahm. Die anschließende Annotation der rund 22,4 Mio. Spiel- und Puzzle-Positionen mit Stockfish bis Tiefe 12 (Abschnitt 3.2) benötigte weitere 18 Stunden. Beide Schritte fallen einmalig an und laufen ohne GPU. Das eigentliche Training einer Epoche über die 469 Chunks dauerte anschließend rund 8 Stunden auf einer Kaggle-GPU (P100).

Blattauswertung und Batch-Größe Zur Laufzeit dominiert die Auswertung der Suchbaum-Blätter durch das Netz die Kosten. Da MCTS die Blätter stapelweise auf der GPU auswertet, wurde die Batch-Größe über 50 Teststellungen bei fester Bedenkzeit von 1,0 s je Zug variiert (Tabelle 8). Der Durchsatz steigt bis Batch 32 auf das rund Zehnfache (108 auf 1147 Simulationen/s), während die Zugqualität, gemessen als mittlerer Centipawn-Verlust gegenüber Stockfish, praktisch konstant bleibt (rund 9,1). Erst bei Batch 64 bricht die Qualität ein (11,3 cp). So viele gleichzeitig ausgewählte Blätter lassen die Suchstatistik veralten, sodass trotz Virtual Loss vermehrt derselbe Ast doppelt expandiert wird. Batch 32 bietet daher den besten Kompromiss und wurde für alle Messungen gewählt.

Batch-Größe	Simulationen/s	Mittl. cp-Verlust
1	108	9,1
8	573	9,2
16	882	9,2
32	1147	9,1
64	1368	11,3

Tabelle 8: Durchsatz und Zugqualität nach Batch-Größe der Blattauswertung.

Kosten je Partie Bei Batch 32 benötigt das Netz für einen Zug (1000 Simulationen) ungefähr 0,75 s. Stockfish wurde daher mit 0,75 s/Zug ungefähr dieselbe Bedenkzeit zugestanden, was einen fairen Zeitvergleich für den Anker sicherstellt (Abschnitt 4.1). Eine vollständige Partie zweier Netze mit je 1000 Simulationen dauert so im Mittel etwa 1,6 Minuten, ein 100-Partie-Match entsprechend rund 2,5 bis 3 Stunden.

Self-Play (RL) Auf der Seite des Reinforcement Learnings dominiert die Datenerzeugung. Jede Trainingsposition entsteht aus einer vollständigen Baumsuche, sodass die Kosten mit der Zahl der Simulationen und der Spiele je Iteration multiplikativ wachsen. Die funktionierende Konfiguration mit 800 Simulationen und 100 Spiele je Iteration kostete rund 2,4 h pro Iteration auf der Kaggle-GPU (P100), wovon der Self-Play mit etwa 96 % den weit überwiegenden Teil ausmachte und Training sowie Gate zusammen unter 5 % blieben. Der Zusatzlauf mit 2000 Spielen bei nur 100 Simulationen verdeutlicht die multiplikative Kostenstruktur, hier stieg der Aufwand auf rund 5 h je Iteration, ohne dass daraus ein Lernsignal entstand (Abschnitt 4.3). Über alle Läufe hinweg, einschließlich

der gescheiterten Frühkonfigurationen sowie der Evaluations- und Anker-Matches, wurden auf derselben GPU rund 120 GPU-Stunden aufgewendet.

5 Diskussion

Einordnung der Ergebnisse Unter den hier gewählten Bedingungen ist der SL-Agent dem RL-Agenten deutlich überlegen. Er erreicht rund 1920 gegenüber rund 1330 Elo und gewinnt das direkte Match mit einem Punktanteil von 0,93 (Abschnitt 4.4). Dieses Ergebnis ist allerdings kein Beleg dafür, dass Self-Play als Verfahren versagt. Gegen sein eigenes Ausgangsnetz gewinnt rl_4 klar und hat dessen Spielstärke um 150 bis 200 Elo angehoben. Das Verfahren lernt also auch in diesem Rahmen, es startete nur von einem deutlich niedrigeren Niveau und konnte den Abstand im verfügbaren Budget nicht aufholen.

Der entscheidende Faktor ist der Rechenaufwand Der Grund für diesen Abstand liegt weniger im Lernverfahren als in den Kosten der Datenbeschaffung. Beide Agenten trainierten auf derselben Hardware, einer Kaggle-GPU vom Typ P100. Das SL-Training benötigte davon rund acht GPU-Stunden, denen eine einmalige, GPU-freie Aufbereitung von etwa 31 Stunden vorausging. Das Reinforcement Learning verbrauchte dagegen rund 120 GPU-Stunden und blieb dabei fast 600 Elo zurück (Abschnitt 4.5). Der Unterschied folgt unmittelbar aus der Herkunft der Daten. Supervised Learning greift auf einen Korpus zu, in dem die Spielstärke von Millionen menschlicher Partien bereits enthalten ist, und muss diesen nur noch aufbereiten. Self-Play muss jede einzelne Trainingsposition selbst erzeugen, und zwar über eine hinreichend starke Baumsuche, denn nur dann liefert die Suche ein besseres Ziel als das Netz selbst und es gibt überhaupt etwas zu lernen (Abschnitt 4.3). Genau diese Bedingung macht die Datenerzeugung teuer und ist unter kleinem Budget der bindende Engpass.

Das Trainingssignal prägt die Schwächen Dass beide Agenten trotz identischer Architektur, Kodierung und Suche an entgegengesetzten Stellen scheitern (Abschnitt 4.4), ist unmittelbar auf die Herkunft ihrer Daten zurückzuführen. Auf der Seite des Supervised Learnings bestätigt sich derselbe Zusammenhang noch einmal im Kleinen. Zwischen v1 und v4 wurden weder Architektur noch Suche verändert, sondern ausschließlich Zusammensetzung und Beschriftung der Daten, und allein daraus entstand der Unterschied zwischen einem Netz, das nicht mattsetzen konnte, und dem stärksten Agenten dieser Arbeit.

Grenzen der Messung Die berichteten Elo-Werte sind Schätzungen mit erheblicher Unsicherheit. Ein Match über 100 Partien erlaubt bestenfalls eine Auflösung von einigen Dutzend Elo, sodass kleinere Unterschiede nicht belastbar sind. Der absolute Anker beruht zudem auf Stockfish mit gedrosseltem UCI_Elo, was eine künstlich geschwächte

Spielweise erzeugt und keine auf menschliche Spieler geeichte Skala darstellt. Für `rl_4` kommt hinzu, dass der Übergang zum ausgeglichenen Match unterhalb der niedrigsten einstellbaren Stufe liegt und der Wert daher extrapoliert bleibt (Abschnitt 4.4).

Grenzen des Vergleichs Auf der Seite des Reinforcement Learnings wurden in der funktionierenden Konfiguration zwei Faktoren gleichzeitig verändert, die Zahl der Spiele und die Wahl eines schwächeren Seeds, sodass ihr jeweiliger Anteil offen bleibt (Abschnitt 4.3). Vor allem aber unterscheiden sich die beiden Ansätze in ihrer oberen Schranke. Supervised Learning ist durch seine Lehrerquelle nach oben begrenzt (Abschnitt 2.6), und diese besteht hier aus Partien menschlicher Spieler und einer Stockfish-Bewertung bis Tiefe 12. Self-Play kennt diese Schranke nicht. Mit wachsendem Rechenbudget verschiebt sich der hier gemessene Vorteil daher voraussichtlich, und die Reihenfolge beider Ansätze ist nicht als allgemeingültig zu verstehen.

6 Fazit

Die Arbeit ist der Frage nachgegangen, welches Trainingssignal den stärkeren Schachagenten liefert, wenn nur studentische Rechenzeit zur Verfügung steht. Unter diesen Bedingungen fällt die Antwort eindeutig aus. Der SL-Agent erreicht rund 1920 Elo und gewinnt das direkte Match gegen den RL-Agenten mit einem Punktanteil von 0,93, wobei er dafür nur einen Bruchteil der Rechenzeit benötigte. Bezogen auf die Titelfrage bleibt unter diesen Bedingungen also der angelernte Agent vorn.

Diese Antwort gilt jedoch unter einer wesentlichen Bedingung. Self-Play hat sein Ausgangsnetz um 150 bis 200 Elo verbessert und damit gezeigt, dass es auch in diesem Rahmen lernt. Sein Nachteil liegt nicht im Lernprinzip, sondern darin, dass es jede Trainingsposition durch eine eigene Suche erzeugen muss, während Supervised Learning auf einem bereits vorhandenen Korpus aufsetzt. Da dieser Korpus zugleich dessen Obergrenze bildet, ist zu erwarten, dass sich das Verhältnis mit wachsendem Rechenbudget umkehrt. Der Code, der in dieser Arbeit produziert und verwendet wurde, kann auf Github [24] und die Demo in einem öffentlichen Kaggle Notebook [25] eingesehen werden.

KI-Offenlegung

Im Rahmen dieser Arbeit wurde das KI-Sprachmodell *Claude Opus 4.8* (Anthropic) unterstützend eingesetzt. Der Einsatz umfasste die Ausformulierung von Textpassagen auf Basis eigener Notizen und Gliederungen sowie die Überarbeitung und Verbesserung bestehender Formulierungen. Darüber hinaus wurde das Modell bei der Entwicklung von Code (GitHub, Kaggle Notebook) sowie bei der initialen Erstellung von PowerPoint-Folien eingesetzt, die anschließend inhaltlich und gestalterisch überarbeitet wurden.

Sämtliche KI-generierten Inhalte wurden kritisch geprüft, überarbeitet und eigenverantwortlich in die Arbeit integriert. Die inhaltliche Verantwortung für alle Aussagen, Ergebnisse und Schlussfolgerungen liegt vollständig bei den Verfassern.

Literatur

- [1] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan und Demis Hassabis. „A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play“. In: *Science* 362.6419 (2018), S. 1140–1144.
- [2] Stockfish Contributors. *Stockfish: A Strong Open Source Chess Engine*. <https://stockfishchess.org>. Abgerufen am 30. Juli 2026. 2026.
- [3] Claude E. Shannon. „Programming a Computer for Playing Chess“. In: *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 41.314 (1950), S. 256–275.
- [4] Jonathan Schaeffer, Neil Burch, Yngvi Bjornsson, Akihiro Kishimoto, Martin Muller, Robert Lake, Paul Lu und Steve Sutphen. „Checkers is solved“. In: *science* 317.5844 (2007), S. 1518–1522.
- [5] Arpad E. Elo. *The Rating of Chess Players, Past and Present*. New York: Arco, 1978.
- [6] Lichess. *Frequently Asked Questions — What is the average centipawn loss (ACPL)?* Lichess. URL: <https://lichess.org/faq#acpl> (besucht am 30.07.2026).
- [7] Matej Guid und Ivan Bratko. „Computer Analysis of World Chess Champions“. In: *ICGA Journal* 29.2 (2006), S. 65–73. DOI: 10.3233/ICG-2006-29203.
- [8] Rafael V. Leite und Anderson V. C. de Oliveira. „Expected Human Performance Behavior in Chess Using Centipawn Loss Analysis“. In: *HCI in Games*. Hrsg. von Xiaowen Fang. Bd. 14047. Lecture Notes in Computer Science. Cham: Springer, 2023, S. 243–252. DOI: 10.1007/978-3-031-35979-8_19.
- [9] Donald E. Knuth und Ronald W. Moore. „An Analysis of Alpha-Beta Pruning“. In: *Artificial Intelligence* 6.4 (1975), S. 293–326.
- [10] Murray Campbell, A. Joseph Hoane und Feng-hsiung Hsu. „Deep Blue“. In: *Artificial Intelligence* 134.1–2 (2002), S. 57–83.
- [11] Dominik Klein. „Neural networks for chess“. In: *arXiv preprint arXiv:2209.01506* (2022).
- [12] Leela Chess Zero. *Neural Network Topology*. 2021. URL: <https://lczero.org/dev/backend/nn/> (besucht am 23.07.2026).
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren und Jian Sun. „Deep Residual Learning for Image Recognition“. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, S. 770–778.

- [14] Jie Hu, Li Shen und Gang Sun. „Squeeze-and-Excitation Networks“. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, S. 7132–7141.
- [15] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel und Demis Hassabis. „Mastering the Game of Go with Deep Neural Networks and Tree Search“. In: *Nature* 529.7587 (2016), S. 484–489.
- [16] Rémi Coulom. „Efficient selectivity and backup operators in Monte-Carlo tree search“. In: *International conference on computers and games*. Springer. 2006, S. 72–83.
- [17] Peter Auer, Nicolo Cesa-Bianchi und Paul Fischer. „Finite-time analysis of the multiarmed bandit problem“. In: *Machine learning* 47.2 (2002), S. 235–256.
- [18] Levente Kocsis und Csaba Szepesvári. „Bandit based monte-carlo planning“. In: *European conference on machine learning*. Springer. 2006, S. 282–293.
- [19] Christopher D Rosin. „Multi-armed bandits with episode context“. In: *Annals of Mathematics and Artificial Intelligence* 61.3 (2011), S. 203–230.
- [20] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton u. a. „Mastering the game of go without human knowledge“. In: *nature* 550.7676 (2017), S. 354–359.
- [21] Leela Chess Zero Contributors. *Convolutional Policy Head*. Diskussion in GitHub-Issue #637. 2019. URL: <https://github.com/LeelaChessZero/lc0/issues/637> (besucht am 23.07.2026).
- [22] Lichess. *Wöchentliche Blitz-Ratingverteilung*. 2026. URL: <https://lichess.org/stat/rating/distribution/blitz> (besucht am 25.07.2026).
- [23] Lichess. *Lichess Accuracy metric*. 2026. URL: <https://lichess.org/page/accuracy> (besucht am 30.07.2026).
- [24] Danny Hoffmann, Tim Lehmann, Joshua Meyer und Philipp Meyer. *chess_bot*. GitHub Repository. 2026. URL: https://github.com/GitWithJosh/chess_bot (besucht am 30.07.2026).
- [25] Danny Hoffmann, Tim Lehmann, Joshua Meyer und Philipp Meyer. *Public ChessNN Projekt WDSKI23*. Kaggle Notebook. 2026. URL: <https://www.kaggle.com/code/dhoffmann000/public-chessnn-projekt-wdski23> (besucht am 30.07.2026).